

Complete Angular Interview Preparation

Zero to Advanced — Theory, Hands-on Code, Interview Q&A, and Video Resources for a 3-year experienced Angular developer aiming for senior roles.

Target reader: Frontend developer with ~3 years of Angular experience preparing for mid-to-senior level interviews at product companies, startups, and enterprise firms.

Angular version: Angular 17+ with modern syntax — `@if`, `@for`, `@defer`, signals, standalone components, new control flow. Legacy syntax noted where relevant.

TypeScript & JavaScript Fundamentals | Angular Core Concepts | Component Architecture | RxJS & State Management | Routing & Navigation | Forms & Validation | HTTP & API Integration | Testing | Performance | Security | Build & Deployment | System Design

1. JavaScript & TypeScript Fundamentals

Angular is built on TypeScript, and TypeScript is a superset of JavaScript. Interviewers expect you to be strong in both languages before they discuss Angular.

Theory

JavaScript runs on a single-threaded event loop. Understanding this is essential — it explains why long-running synchronous code blocks the UI and why async patterns (callbacks, promises, `async/await`, observables) are critical. TypeScript adds a type system that catches bugs at compile time and powers Angular's dependency injection, decorators, and template type-checking.

1.1 JavaScript essentials you must explain confidently

Topic	What to know	Why interviewers ask
Event loop	Call stack, task queue, microtask queue, <code>requestAnimationFrame</code>	Explains async behavior, <code>setTimeout</code> vs Promise ordering
Closures	A function retains access to its outer scope even after the outer function returns	Used in RxJS operators, factory functions, Angular services
Prototypal inheritance	Objects inherit from other objects via the prototype chain	Explains how class extends works under the hood
this keyword	Depends on how the function is called: direct, method, constructor, arrow, <code>bind/call/apply</code>	Arrow functions in callbacks, Angular event handlers
Hoisting	<code>var</code> is hoisted (undefined), <code>let/const</code> are in TDZ, functions are fully hoisted	Scope bugs, debugging unexpected undefined values
ES6+ features	destructuring, spread/rest, template literals, optional chaining, nullish coalescing, modules	Modern Angular code uses all of these daily
Promises & <code>async/await</code>	Promise states, chaining, error handling, <code>Promise.all/race/allSettled</code>	HTTP calls, resolver patterns, sequential vs parallel async
Map, Set, WeakMap, WeakRef	When to use each, garbage collection behavior	Caching patterns, state management edge cases

1.2 TypeScript essentials

Topic	What to know	Angular use case
Interfaces vs Types	Interfaces are for object shapes and are extendable. Types support unions, intersections, mapped types	Component inputs, API models, service contracts
Generics	Type parameters for reusable, type-safe functions, classes, and interfaces	HttpClient<T>, custom services, RxJS operators
Decorators	Functions that modify class/property/method behavior at design time	@Component, @Injectable, @Input, @Output, custom decorators
Enums	Named constants. Prefer const enum or string union types for tree-shaking	API status codes, form field types, route constants
Utility types	Partial, Required, Pick, Omit, Record, Readonly, ReturnType, Parameters	Form models, API payloads, component prop subsets
Strict mode	strictNullChecks, strictPropertyInitialization, noImplicitAny	Angular CLI default since v12, catches bugs early
Type guards & narrowing	typeof, instanceof, in, custom type predicates	Handling union types in templates and services
Module systems	ES modules (import/export), CommonJS, dynamic import()	Lazy loading routes, barrel exports, tree shaking

Hands-on Code

Event loop — predict the output

```
console.log('1');

setTimeout(() => console.log('2'), 0);

Promise.resolve().then(() => console.log('3'));

console.log('4');

// Output: 1, 4, 3, 2
// Why: synchronous code runs first (1, 4),
// then microtask queue (Promise → 3),
// then macrotask queue (setTimeout → 2)
```

Theory: Microtasks (promises, MutationObserver) run before macrotasks (setTimeout, setInterval, I/O). This is why Angular's change detection triggered by Zone.js (which patches both) can intercept async completions in a predictable order.

Closures — practical Angular example

```
// Factory function using closure – common in Angular services
function createCounter(initial: number) {
  let count = initial; // closed over

  return {
```

```

    increment: () => ++count,
    decrement: () => --count,
    getCount: () => count
  };
}

const counter = createCounter(10);
counter.increment(); // 11
counter.increment(); // 12
counter.getCount(); // 12
// The variable 'count' survives because the returned object's methods
// close over it. This is exactly how Angular singleton services maintain state.

```

TypeScript generics — real-world service

```

// Generic API service with type safety
@Injectable({ providedIn: 'root' })
export class ApiService {
  constructor(private http: HttpClient) {}

  get<T>(url: string, params?: Record<string, string>): Observable<T> {
    return this.http.get<T>(url, { params });
  }

  post<TReq, TRes>(url: string, body: TReq): Observable<TRes> {
    return this.http.post<TRes>(url, body);
  }
}

// Usage – compiler knows the return type
interface User { id: number; name: string; email: string; }
this.api.get<User[]>('/api/users').subscribe(users => {
  // users is User[], not any
  console.log(users[0].name);
});

```

TypeScript utility types — form models

```

interface User {
  id: number;
  name: string;
  email: string;
  password: string;
  createdAt: Date;
}

// For an edit form, id and createdAt are read-only, password is optional
type EditUserForm = Omit<User, 'id' | 'createdAt'> & {
  password?: string;
};

// For API create payload – no id yet
type CreateUserPayload = Omit<User, 'id' | 'createdAt'>;

// For read-only display
type UserDisplay = Readonly<Pick<User, 'id' | 'name' | 'email'>>;

```

Interview Questions

Q: What is the difference between var, let, and const?

A: **var** is function-scoped, hoisted to the top of its function as **undefined**, and can be redeclared. **let** is block-scoped, hoisted but in the temporal dead zone until declaration, and cannot be redeclared. **const** is block-scoped like **let** but the binding cannot be reassigned (the value itself can still be mutated if it is an object or array).

Q: Explain closures with a real example.

A: A closure is a function that retains access to its outer scope's variables even after the outer function has returned. In Angular, every service created by a factory function is a closure — the service methods close over the injected dependencies and private state. RxJS operators like `map` and `filter` also use closures to capture the projection/predicate function.

Q: What is the difference between interface and type in TypeScript?

A: Interfaces are for object shapes — they support declaration merging, extending, and are used by Angular's DI system. Types support unions (`string | number`), intersections (`A & B`), mapped types, conditional types, and template literal types. Rule of thumb: use interfaces for API models and component contracts, use types for unions, computed types, and complex type algebra.

Q: What are generics and why do they matter?

A: Generics let you write type-safe code that works with multiple types without losing type information. Without generics, `HttpClient.get()` would return `Observable<any>` and you'd lose autocomplete and type checking. With generics, `HttpClient.get<User[]>('/api/users')` returns `Observable<User[]>` and the compiler knows exactly what properties are available.

Q: How does the JavaScript event loop work?

A: JavaScript has a single call stack. When the stack is empty, the engine checks the microtask queue first (promises, `queueMicrotask`), running all microtasks. Then it checks the macrotask queue (`setTimeout`, events, I/O) and runs one macrotask. Then it checks microtasks again. This cycle repeats. Angular's `Zone.js` patches both queues to detect when async work completes, triggering change detection.

Video Resources

- [What the heck is the event loop anyway? — Philip Roberts](#)
- [TypeScript Full Course for Beginners \(English\)](#)
- [JavaScript Namaste Series by Akshay Saini \[Hindi\]](#)
- [TypeScript Tutorial in Hindi](#)

2. Angular Core Architecture

Theory

Angular is a platform and framework for building client applications in HTML, CSS, and TypeScript. It provides a structured runtime that manages component lifecycle, dependency injection, change detection, routing, forms, HTTP, and more. Understanding the architecture is the foundation of every Angular interview.

2.1 Key architecture concepts

Concept	What it is	Why it matters
Modules (NgModule)	Containers that group components, directives, pipes, and services. Being replaced by standalone components in modern Angular.	Understanding imports, declarations, providers, and the transition to standalone
Standalone Components	Components that declare their own dependencies via <code>imports</code> array — no NgModule needed	Default since Angular 17. Simpler mental model, better tree-shaking
Components	Building blocks with a template, class, styles, and metadata	Every UI element is a component — understanding their lifecycle is critical
Templates	HTML + Angular syntax: bindings, directives, pipes, control flow	Templates are compiled to JavaScript instructions — they are not HTML strings
Directives	Structural (<code>@if</code> , <code>@for</code> , <code>*ngIf</code>), attribute, and custom directives	Extend HTML behavior, reuse UI logic
Pipes	Transform displayed values in templates — pure and impure	Pure pipes are memoized; impure pipes re-run on every CD cycle
Services & DI	Singleton or scoped classes injected via constructor or <code>inject()</code>	Angular's DI is hierarchical — provider scope affects data sharing
Dependency Injection	A design pattern where classes receive dependencies instead of creating them	Testability, modularity, and single-responsibility principle

2.2 Component lifecycle hooks

Hook	When it runs	Common use
<code>ngOnChanges</code>	When any <code>@Input</code> changes (before <code>ngOnInit</code> on first run)	React to specific input changes, compare previous/current values
<code>ngOnInit</code>	Once, after first <code>ngOnChanges</code>	Initial data fetching, setup logic that depends on inputs
<code>ngDoCheck</code>	Every change detection cycle	Custom change detection logic (use sparingly — it runs very often)

Hook	When it runs	Common use
ngAfterContentInit	Once, after content projection completes	Access projected content via @ViewChild
ngAfterContentChecked	After every check of projected content	Rarely needed — use with caution
ngAfterViewInit	Once, after view and children are initialized	Access child components/elements via @ViewChild, init charts/maps
ngAfterViewChecked	After every check of the view	Rarely needed — use with caution
ngOnDestroy	Just before Angular destroys the component	Unsubscribe, clear timers, remove listeners, clean up resources

2.3 Dependency Injection — deep theory

Angular's DI system is a hierarchical injector tree that mirrors the component tree. When a component requests a dependency, Angular walks up the injector tree until it finds a provider.

- **providedIn: 'root'** — singleton across the entire app. Tree-shakeable.
- **providedIn: 'any'** — one instance per lazy-loaded module boundary.
- **Component-level providers** — new instance per component. Useful for services with component-scoped state.
- **Injection tokens** — InjectionToken for non-class values (config objects, feature flags).
- **inject() function** — modern alternative to constructor injection. Works in constructors, field initializers, and factory functions.

Hands-on Code

Standalone component with modern syntax

```
import { Component, signal, computed } from '@angular/core';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-greeting',
  standalone: true,
  imports: [CommonModule],
  template: `
    <h2>Hello, {{ name() }}!</h2>
    <p>Character count: {{ charCount() }}</p>
    <input [value]="name()" (input)="name.set($any($event.target).value)" />
  `
})
export class GreetingComponent {
  name = signal('Angular Developer');
  charCount = computed(() => this.name().length);
}
```

Dependency Injection — service scoping

```
// Root-level singleton
@Injectable({ providedIn: 'root' })
export class AuthService {
  private currentUser = signal<User | null>(null);
  readonly user = this.currentUser.asReadonly();

  login(credentials: Credentials): Observable<User> {
    return this.http.post<User>('/api/login', credentials).pipe(
```

```

        tap(user => this.currentUser.set(user))
    );
}
}

// Component-scoped service – new instance per component
@Injectable()
export class FormStateService {
    private dirty = signal(false);
    readonly isDirty = this.dirty.asReadonly();
    markDirty() { this.dirty.set(true); }
}

@Component({
    providers: [FormStateService], // new instance per component
    template: `...`
})
export class EditUserComponent {
    formState = inject(FormStateService);
}

```

Lifecycle hooks — practical example

```

@Component({
    selector: 'app-user-profile',
    template: `
        @if (user()) {
            <h2>{{ user()!.name }}</h2>
            <canvas #chart></canvas>
        }
    `
})
export class UserProfileComponent implements OnInit, AfterViewInit, OnDestroy {
    @Input({ required: true }) userId!: number;
    @ViewChild('chart') chartCanvas!: ElementRef<HTMLCanvasElement>;

    user = signal<User | null>(null);
    private destroyRef = inject(DestroyRef);
    private userService = inject(UserService);
    private chartInstance?: Chart;

    // ngOnInit – fetch data that depends on inputs
    ngOnInit() {
        this.userService.getUser(this.userId).pipe(
            takeUntilDestroyed(this.destroyRef)
        ).subscribe(user => this.user.set(user));
    }

    // ngAfterViewInit – DOM is ready, safe to init chart
    ngAfterViewInit() {
        if (this.chartCanvas) {
            this.chartInstance = new Chart(this.chartCanvas.nativeElement, { /*...*/ });
        }
    }

    // ngOnDestroy – cleanup
    ngOnDestroy() {
        this.chartInstance?.destroy();
    }
}

```

Interview Questions

Q: What is the difference between NgModule and standalone components?

A: NgModules group components, directives, pipes, and providers into cohesive blocks. Standalone components declare their own dependencies via the `imports` array and don't need an NgModule. Standalone is the default since

Angular 17. Benefits: simpler mental model, better tree-shaking (unused standalone components are removed from the build), and easier lazy loading (individual components can be lazy-loaded, not just modules).

Q: Explain Angular's dependency injection hierarchy.

A: Angular maintains a tree of injectors that mirrors the component tree. Root injector → module injectors → component injectors. When a component requests a service, Angular searches upward. `providedIn: 'root'` registers at the root injector (singleton). Component-level `providers` creates a new instance per component instance. This hierarchy enables scoped services, component-level state, and lazy module isolation.

Q: When do you use `ngOnInit` vs constructor?

A: Constructor is for DI — injecting dependencies. It runs before Angular sets inputs and before the component is fully initialized. `ngOnInit` runs after the first `ngOnChanges`, meaning inputs are available. Use `ngOnInit` for initialization logic that depends on inputs, API calls, and setup that should only run once. In modern Angular, the `inject()` function in field initializers can replace some constructor patterns.

Q: What is the difference between `@ViewChild` and `@ContentChild`?

A: `@ViewChild` accesses elements/components defined in the component's own template. `@ContentChild` accesses elements/components projected via `<ng-content>` from the parent. `@ViewChild` is available in `ngAfterViewInit`. `@ContentChild` is available in `ngAfterContentInit`.

Q: Explain pure vs impure pipes.

A: A pure pipe (default) only re-executes when the input value reference changes — Angular caches the result. An impure pipe (`pure: false`) re-executes on every change detection cycle, regardless of whether inputs changed. Pure pipes are performant. Impure pipes (like `async`) are expensive but necessary when the pipe needs to react to changes that don't alter the reference (like array mutations).

Video Resources

- [Angular Official Overview and Documentation](#)
- [Angular Course 2024 — Complete Guide \(Maximilian Schwarzmüller\)](#)
- [Angular Full Tutorial — Codevolution](#)
- [Angular Tutorial in Hindi \(Code Step By Step\)](#)

3. Component Communication & Architecture Patterns

Theory

Real Angular applications are trees of components that must share data and coordinate behavior. How you pass data determines the maintainability and scalability of your app. There are multiple communication patterns, each suited to different scenarios.

3.1 Communication methods

Pattern	Direction	When to use
@Input / @Output	Parent ↔ Child	Direct parent-child data flow. Most common pattern.
Signals (input/output/model)	Parent ↔ Child	Modern Angular 17+ alternative. Signal-based inputs with transform support.
Content projection (ng-content)	Parent → Child	Flexible, reusable wrappers — card, modal, layout components
@ViewChild / @ContentChild	Parent → Child	Imperative access to child component API (use sparingly)
Services with signals/subjects	Any ↔ Any	Sibling communication, cross-tree shared state
Router (params, data, resolve)	Route → Component	Page-level state, deeplinkable state
Signal Store / NgRx	Global	Large apps needing predictable, centralized state management

Hands-on Code

Modern signal-based input/output

```
// Parent component
@Component({
  template: `
    <app-product-card
      [product]="selectedProduct()"
      (addToCart)="onAddToCart($event)"
    />
  `,
})
export class ShopComponent {
  selectedProduct = signal<Product>({ id: 1, name: 'Laptop', price: 999 });

  onAddToCart(product: Product) {
    this.cartService.add(product);
  }
}

// Child component – signal input (Angular 17+)
@Component({
  selector: 'app-product-card',
  template: `
    <div class="card">
      <h3>{{ product().name }}</h3>
      <p>■{{ product().price | number }}</p>
    </div>
  `
})
```

```

        <button (click)="addToCart.emit(product())">Add to Cart</button>
      </div>
    ,
  })
  export class ProductCardComponent {
    product = input.required<Product>(); // signal input
    addToCart = output<Product>(); // signal output
  }

```

Content projection — reusable card component

```

// Reusable card wrapper
@Component({
  selector: 'app-card',
  template: `
    <div class="card">
      <div class="card-header">
        <ng-content select="[card-title]"></ng-content>
      </div>
      <div class="card-body">
        <ng-content></ng-content>
      </div>
      <div class="card-footer">
        <ng-content select="[card-actions]"></ng-content>
      </div>
    </div>
  `
})
export class CardComponent {}

// Usage – multiple named slots
<app-card>
  <h3 card-title>User Profile</h3>
  <p>Name: {{ user.name }}</p>
  <p>Email: {{ user.email }}</p>
  <button card-actions (click)="edit()">Edit</button>
</app-card>

```

Service-based sibling communication

```

// Shared state service using signals
@Injectable({ providedIn: 'root' })
export class NotificationService {
  private _notifications = signal<Notification[]>([]);
  readonly notifications = this._notifications.asReadonly();
  readonly unreadCount = computed(() =>
    this._notifications().filter(n => !n.read).length
  );

  add(message: string, type: 'info' | 'error' | 'success') {
    this._notifications.update(list => [
      ...list,
      { id: crypto.randomUUID(), message, type, read: false, timestamp: new Date() }
    ]);
  }

  markRead(id: string) {
    this._notifications.update(list =>
      list.map(n => n.id === id ? { ...n, read: true } : n)
    );
  }
}

// Any component can now inject and use:
// Navbar reads unreadCount, NotificationPanel reads notifications
// Neither component knows about the other

```

Interview Questions

Q: When would you use a service for communication vs @Input/@Output?

A: @Input/@Output is for direct parent-child relationships. For sibling components, deeply nested components, or components in different branches of the tree, use a shared service. Services are also better when the state needs to persist across route navigations. Rule: if you are passing data through more than 2 intermediate components (prop drilling), switch to a service.

Q: What is content projection and when do you use it?

A: Content projection (<ng-content>) lets a parent component pass template content into a child component. Use it for reusable layout components — cards, modals, dialogs, panels, tabs — where the structure is fixed but the content varies. Multi-slot projection with `select` lets you target different content areas. It is the Angular equivalent of React's children/slots.

Q: Explain the smart/dumb (container/presentational) component pattern.

A: Smart components (containers) handle data fetching, state management, and business logic. Dumb components (presentational) receive data via @Input and emit events via @Output — they have no knowledge of services or state. Benefits: dumb components are highly reusable, easy to test, and easy to reason about. Smart components coordinate the page-level logic. This separation makes the codebase more maintainable as it grows.

4. Change Detection & Rendering

Theory

Change detection is Angular's mechanism for keeping the view in sync with the component state. Understanding how it works is one of the most common interview topics because it directly impacts performance.

4.1 How change detection works

Angular walks the component tree top-down, checking each component's template bindings. If a binding's value has changed, Angular updates the DOM. This process is triggered by Zone.js, which patches all async APIs (events, timers, HTTP) so Angular knows when to check.

4.2 Default vs OnPush strategies

Feature	Default	OnPush
When it checks	Every change detection cycle	Only when inputs change by reference, events fire inside, or manually triggered
Performance	Convenient but can be expensive in large trees	Efficient — skips subtrees when not needed
Data patterns	Works with any mutation pattern	Requires immutable data / signal updates
When to use	Small apps, prototyping, leaf components	All components in medium-to-large apps

4.3 Signals — the future of change detection

Signals are Angular's fine-grained reactivity primitive. Instead of Zone.js triggering broad CD cycles, signals create a dependency graph where only the specific consumers that read a changed signal are updated. This is more precise and more efficient than tree-based checking.

Hands-on Code

Default vs OnPush — side-by-side comparison

```
// DEFAULT — this template expression runs on EVERY CD cycle
@Component({
  template: `<p>{{ getFullName() }}</p>` // BAD — method in template
})
export class DefaultComponent {
  user = { first: 'Sid', last: 'Rameshwar' };
  getFullName() {
    console.log('getFullName called'); // Runs 20+ times per second!
    return `${this.user.first} ${this.user.last}`;
  }
}

// ONPUSH + SIGNALS — computed runs only when dependencies change
@Component({
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `<p>{{ fullName() }}</p>`
})
export class OnPushComponent {
  user = signal({ first: 'Sid', last: 'Rameshwar' });
  fullName = computed(() => {
    console.log('fullName computed'); // Runs only when user signal changes
    return `${this.user().first} ${this.user().last}`;
  });
}
```

```
    });
  }
}
```

Zone.js — running outside Angular

```
@Component({
  template: `<canvas #canvas></canvas>`
})
export class AnimationComponent implements AfterViewInit {
  @ViewChild('canvas') canvas!: ElementRef<HTMLCanvasElement>;
  private ngZone = inject(NgZone);

  ngAfterViewInit() {
    // This animation loop does NOT trigger change detection
    this.ngZone.runOutsideAngular(() => {
      this.animate();
    });
  }

  private animate() {
    const ctx = this.canvas.nativeElement.getContext('2d')!;
    // ... draw frame ...
    requestAnimationFrame(() => this.animate());
    // Without runOutsideAngular, each rAF frame would trigger CD!
  }
}
```

Interview Questions

Q: What triggers change detection in Angular?

A: Zone.js patches async APIs and triggers CD when they complete: (1) DOM events (click, input, scroll), (2) Timers (setTimeout, setInterval), (3) HTTP responses, (4) Promise resolutions and microtasks. With OnPush + signals, CD still needs a trigger, but the framework can skip unchanged subtrees. Zoneless Angular relies entirely on signal-based reactivity.

Q: What is the difference between signals and observables?

A: Signals are synchronous, hold a current value, and create a dependency graph — they are for state. Observables are asynchronous streams of values over time — they are for events and async data flows. Signals auto-notify consumers when their value changes. Observables require explicit subscription. In Angular, signals are used for component state and template bindings; observables (RxJS) are used for HTTP, WebSockets, user event streams, and complex async orchestration.

5. RxJS Deep Dive

Theory

RxJS (Reactive Extensions for JavaScript) is Angular's async engine. It provides **observables** — lazy, composable, cancellable streams of values. Understanding RxJS is non-negotiable for Angular interviews.

5.1 Core concepts

- **Observable:** a lazy collection of values over time. Does not execute until subscribed.
- **Observer:** an object with `next`, `error`, `complete` callbacks.
- **Subscription:** represents the execution of an observable. Must be cleaned up.
- **Operators:** pure functions that transform, filter, combine, and control streams.
- **Subject:** both observable and observer. Can multicast values to many subscribers.
- **BehaviorSubject:** Subject that holds a current value and emits it to new subscribers.
- **ReplaySubject:** Caches N previous values and replays them to new subscribers.

5.2 Operators you must know

Category	Operators	Use case
Transformation	<code>map</code> , <code>switchMap</code> , <code>mergeMap</code> , <code>concatMap</code> , <code>exhaustMap</code>	Transform values, handle inner observables
Filtering	<code>filter</code> , <code>distinctUntilChanged</code> , <code>debounceTime</code> , <code>throttleTime</code> , <code>take</code> , <code>first</code>	Reduce noise, limit emissions
Combination	<code>combineLatest</code> , <code>forkJoin</code> , <code>merge</code> , <code>zip</code> , <code>withLatestFrom</code>	Combine multiple streams
Error handling	<code>catchError</code> , <code>retry</code> , <code>retryWhen</code>	Graceful error recovery
Utility	<code>tap</code> , <code>delay</code> , <code>finalize</code> , <code>shareReplay</code> , <code>startWith</code>	Side effects, debugging, caching

5.3 The flattening operators — critical interview topic

Operator	Behavior	When to use
<code>switchMap</code>	Cancels previous inner observable when a new outer value arrives	Search autocomplete, route param changes — always want latest
<code>mergeMap</code>	Runs all inner observables concurrently	Parallel file uploads, batch operations where order doesn't matter
<code>concatMap</code>	Queues inner observables — runs them sequentially	Sequential API calls, ordered operations
<code>exhaustMap</code>	Ignores new outer values while inner is still running	Login button, payment submit — prevent double-submit

Hands-on Code

Search autocomplete — the classic RxJS interview question

```
@Component({
  template: `
    <input [formControl]="searchControl" placeholder="Search..." />
    @for (result of results(); track result.id) {
      <div class="result">{{ result.name }}</div>
    }
    @if (loading()) { <div class="spinner">Loading...</div> }
  `
})
export class SearchComponent {
  searchControl = new FormControl('');
  results = signal<SearchResult[]>([]);
  loading = signal(false);
  private destroyRef = inject(DestroyRef);
  private http = inject(HttpClient);

  constructor() {
    this.searchControl.valueChanges.pipe(
      debounceTime(300), // Wait for user to stop typing
      distinctUntilChanged(), // Skip if same value
      tap(() => this.loading.set(true)),
      switchMap(term => // Cancel previous request
        term && term.length > 2
          ? this.http.get<SearchResult[]>(`/api/search?q=${encodeURIComponent(term)}`).pipe(
              catchError(() => of([])) // Handle errors gracefully
            )
          : of([])
      ),
      tap(() => this.loading.set(false)),
      takeUntilDestroyed(this.destroyRef)
    ).subscribe(results => this.results.set(results));
  }
}
```

Theory: This is the most commonly asked RxJS code in interviews. `debounceTime` prevents API calls on every keystroke. `distinctUntilChanged` skips duplicate values. `switchMap` cancels the previous request when a new term arrives — preventing race conditions. `catchError` returns a fallback empty array instead of breaking the stream. `takeUntilDestroyed` prevents memory leaks.

Combining multiple API calls

```
// Parallel requests – both must complete
const dashboard$ = forkJoin({
  user: this.http.get<User>('/api/user'),
  orders: this.http.get<Order[]>('/api/orders'),
  notifications: this.http.get<Notification[]>('/api/notifications')
});
// Emits one object { user, orders, notifications } when ALL complete

// Combined streams – emit whenever any source changes
const display$ = combineLatest({
  selectedProduct: this.productService.selected$,
  cart: this.cartService.items$,
  userPrefs: this.prefsService.preferences$
}).pipe(
  map(({ selectedProduct, cart, userPrefs }) => ({
    product: selectedProduct,
    isInCart: cart.some(item => item.id === selectedProduct.id),
    currency: userPrefs.currency
  }))
);
```

exhaustMap — preventing double-submit

```
// Login form – ignore rapid clicks while request is in flight
```

```

this.loginClicks$.pipe(
  exhaustMap(() =>
    this.authService.login(this.loginForm.value).pipe(
      catchError(err => {
        this.errorMessage.set(err.message);
        return EMPTY;
      })
    )
  )
).subscribe(user => this.router.navigate(['/dashboard']));

```

Interview Questions

Q: What is the difference between switchMap, mergeMap, concatMap, and exhaustMap?

A: All four map an outer value to an inner observable, but they differ in concurrency: **switchMap** cancels previous (use for search). **mergeMap** runs all concurrently (use for parallel uploads). **concatMap** queues sequentially (use for ordered operations). **exhaustMap** ignores new values while busy (use for submit buttons). Choosing wrong causes race conditions or duplicate requests.

Q: How do you prevent memory leaks with RxJS in Angular?

A: (1) Use `async pipe` — it auto-unsubscribes. (2) Use `takeUntilDestroyed()` for imperative subscriptions. (3) Use `take(1)` or `first()` for one-time operations. (4) Store subscriptions and call `.unsubscribe()` in `ngOnDestroy`. (5) Use `DestroyRef` with `onDestroy` callback. The most common leak: subscribing in a component without cleaning up when the component is destroyed.

Q: When do you use Subject vs BehaviorSubject vs ReplaySubject?

A: Subject — multicasts values to current subscribers only, no initial value. **BehaviorSubject** — requires initial value, emits current value to new subscribers immediately. Use for state that always has a value (current user, selected item). **ReplaySubject** — caches N past values and replays them, no initial value required. Use when late subscribers need historical values.

Video Resources

- [RxJS Top Ten Operators — Fireship](#)
- [RxJS Complete Course — Academind](#)
- [RxJS Official Documentation](#)

6. Routing & Navigation

Theory

Angular's router maps URLs to components, enables navigation, and supports lazy loading, guards, resolvers, and nested routes. It is single-page navigation — the page never fully reloads.

6.1 Key routing concepts

- **Route configuration:** maps URL paths to components with optional data, guards, and resolvers.
- **Router outlet:** placeholder where routed components are rendered.
- **Lazy loading:** defer loading of feature routes until the user navigates to them.
- **Guards:** `canActivate`, `canDeactivate`, `canMatch`, `resolve` — control access and data loading.
- **Functional guards (Angular 15+):** plain functions instead of class-based guards.
- **Nested routes:** child routes render inside parent's `<router-outlet>`.
- **Route parameters:** path params (`:id`), query params, and route data.

Hands-on Code

Complete route configuration with guards and lazy loading

```
export const routes: Routes = [
  { path: '', redirectTo: '/dashboard', pathMatch: 'full' },
  {
    path: 'dashboard',
    loadComponent: () => import('./dashboard/dashboard.component')
      .then(m => m.DashboardComponent),
    canActivate: [authGuard]
  },
  {
    path: 'products',
    loadChildren: () => import('./products/product.routes')
      .then(m => m.PRODUCT_ROUTES)
  },
  {
    path: 'admin',
    loadChildren: () => import('./admin/admin.routes').then(m => m.ADMIN_ROUTES),
    canActivate: [authGuard, roleGuard('admin')]
  },
  { path: '**', component: NotFoundComponent }
];

// Functional guard (modern Angular)
export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);

  if (auth.isLoggedIn()) {
    return true;
  }
  return router.createUrlTree(['/login'], {
    queryParams: { returnUrl: state.url }
  });
};

// Role-based guard factory
export function roleGuard(requiredRole: string): CanActivateFn {
  return () => {
    const auth = inject(AuthService);
    return auth.hasRole(requiredRole);
  };
};
```

```
}
```

Reading route params reactively

```
@Component({
  template: `
    @if (product(); as p) {
      <h2>{{ p.name }}</h2>
      <p>{{ p.description }}</p>
    } @else {
      <p>Loading product...</p>
    }
  `
})
export class ProductDetailComponent {
  private route = inject(ActivatedRoute);
  private productService = inject(ProductService);

  product = toSignal(
    this.route.paramMap.pipe(
      map(params => params.get('id')!),
      switchMap(id => this.productService.getById(id))
    )
  );
}
// Theory: route.paramMap is an observable that emits whenever the URL param changes.
// switchMap ensures the previous product request is cancelled when the user
// navigates to a different product. toSignal converts the observable to a signal
// for template use.
```

Interview Questions

Q: What is the difference between canActivate and canMatch?

A: canActivate prevents navigation to a route but the route is still matched — the user sees a redirect. canMatch prevents the route from even being considered during URL matching — useful when you want different components for the same URL based on user role (admin sees admin dashboard, user sees user dashboard).

Q: How does lazy loading work in Angular?

A: Lazy loading uses dynamic import () to defer downloading a module or component until the user navigates to that route. Angular's build system creates separate JavaScript chunks for lazy routes. On navigation, the router triggers the import, the chunk is downloaded, parsed, and the component is rendered. This reduces the initial bundle size and improves startup performance.

Q: How do you handle unsaved changes in a form when the user navigates away?

A: Use a canDeactivate guard that checks if the form is dirty and shows a confirmation dialog. The guard returns true (allow navigation), false (block), or a UrlTree (redirect). This prevents accidental data loss when users navigate away from a half-filled form.

7. Forms — Template-driven & Reactive

Theory

Feature	Template-driven	Reactive
Setup	Directives in template (ngModel)	FormGroup/FormControl in class
Data flow	Two-way binding (async)	Synchronous, immutable
Validation	Directives (required, minlength)	Functions in class
Testability	Harder — requires DOM	Easier — test class logic directly
Dynamic forms	Difficult	Easy — FormArray, programmatic controls
When to use	Simple forms, quick prototypes	Complex forms, dynamic fields, validation logic

Hands-on Code

Reactive form with typed controls, validation, and dynamic fields

```
@Component({
  template: `
    <form [formGroup]="form" (ngSubmit)="onSubmit()">
      <label>Name
        <input formControlName="name" />
        @if (form.controls.name.errors?.['required'] && form.controls.name.touched) {
          <span class="error">Name is required</span>
        }
      </label>

      <label>Email
        <input formControlName="email" type="email" />
        @if (form.controls.email.errors?.['email'] && form.controls.email.touched) {
          <span class="error">Invalid email</span>
        }
      </label>

      <label>Password
        <input formControlName="password" type="password" />
        @if (form.controls.password.errors?.['minlength'] && form.controls.password.touched) {
          <span class="error">Minimum 8 characters</span>
        }
      </label>

      <!-- Dynamic addresses -->
      <h3>Addresses</h3>
      @for (address of addresses.controls; track $index) {
        <div [formGroupName]="$index">
          <input formControlName="street" placeholder="Street" />
          <input formControlName="city" placeholder="City" />
          <button type="button" (click)="removeAddress($index)">Remove</button>
        </div>
      }
      <button type="button" (click)="addAddress()">+ Add Address</button>

      <button type="submit" [disabled]="form.invalid">Register</button>
    </form>
  `,
})
```

```

export class RegistrationComponent {
  private fb = inject(NonNullableFormBuilder);

  form = this.fb.group({
    name: ['', [Validators.required, Validators.minLength(2)]],
    email: ['', [Validators.required, Validators.email]],
    password: ['', [Validators.required, Validators.minLength(8)]],
    addresses: this.fb.array([this.createAddress()])
  });

  get addresses() { return this.form.controls.addresses; }

  private createAddress() {
    return this.fb.group({
      street: ['', Validators.required],
      city: ['', Validators.required]
    });
  }

  addAddress() { this.addresses.push(this.createAddress()); }
  removeAddress(i: number) { this.addresses.removeAt(i); }

  onSubmit() {
    if (this.form.valid) {
      const value = this.form.getRawValue(); // Fully typed!
      console.log(value);
    }
  }
}

```

Custom async validator

```

// Check if email is already taken – async validator
export function emailTakenValidator(http: HttpClient): AsyncValidatorFn {
  return (control: AbstractControl): Observable<ValidationErrors | null> => {
    if (!control.value) return of(null);

    return http.get<{ taken: boolean }>(`/api/check-email?email=${encodeURIComponent(control.value)}`)
      .pipe(
        map(res => res.taken ? { emailTaken: true } : null),
        catchError(() => of(null)) // Don't block form on API error
      );
  };
}

// Usage:
email: ['', [Validators.required, Validators.email], [emailTakenValidator(this.http)]]

```

Interview Questions

Q: When do you choose reactive forms over template-driven?

A: Reactive forms for: complex validation, dynamic fields (FormArray), cross-field validation, forms that need to be tested without DOM, and forms where you need programmatic control. Template-driven for: very simple forms with 2-3 fields, quick prototypes, and forms where two-way binding is sufficient.

Q: What is a custom validator and how do you create one?

A: A custom validator is a function that receives a control and returns either `null` (valid) or a `ValidationErrors` object (invalid). Sync validators return immediately. Async validators return an `Observable` or `Promise` — useful for server-side validation like checking email uniqueness. Custom validators keep validation logic testable and reusable.

Q: How does FormArray work?

A: `FormArray` is an array of `FormControls` or `FormGroups` — used for dynamic, repeatable form sections (addresses, phone numbers, line items). You can push, removeAt, and iterate over entries dynamically. Each entry has its own

validation. It is the standard pattern for "add another" UI patterns.

8. HTTP & API Integration

Theory

Angular's `HttpClient` is a service built on RxJS observables. It provides typed HTTP calls, interceptors for cross-cutting concerns, and supports JSON by default. Interceptors are one of the most powerful and frequently tested patterns.

Hands-on Code

Functional interceptors (modern Angular)

```
// Auth interceptor – adds JWT token to every request
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const token = auth.getToken();

  if (token) {
    req = req.clone({
      setHeaders: { Authorization: `Bearer ${token}` }
    });
  }

  return next(req);
};

// Error interceptor – global error handling
export const errorInterceptor: HttpInterceptorFn = (req, next) => {
  const router = inject(Router);

  return next(req).pipe(
    catchError((error: HttpErrorResponse) => {
      if (error.status === 401) {
        router.navigate(['/login']);
      } else if (error.status === 403) {
        router.navigate(['/forbidden']);
      }
      return throwError(() => error);
    })
  );
};

// Loading interceptor – track active requests
export const loadingInterceptor: HttpInterceptorFn = (req, next) => {
  const loading = inject>LoadingService);
  loading.start();

  return next(req).pipe(
    finalize(() => loading.stop())
  );
};

// Register in app config
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(
      withInterceptors([authInterceptor, errorInterceptor, loadingInterceptor])
    )
  ]
};
```

CRUD service with error handling

```
@Injectable({ providedIn: 'root' })
export class ProductService {
  private http = inject(HttpClient);
```

```

private apiUrl = '/api/products';

getAll(): Observable<Product[]> {
  return this.http.get<Product[]>(this.apiUrl);
}

findById(id: string): Observable<Product> {
  return this.http.get<Product>(`${this.apiUrl}/${id}`);
}

create(product: CreateProductDto): Observable<Product> {
  return this.http.post<Product>(this.apiUrl, product);
}

update(id: string, changes: Partial<Product>): Observable<Product> {
  return this.http.patch<Product>(`${this.apiUrl}/${id}`, changes);
}

delete(id: string): Observable<void> {
  return this.http.delete<void>(`${this.apiUrl}/${id}`);
}
}

```

Interview Questions

Q: What are HTTP interceptors and what are common use cases?

A: Interceptors are middleware that intercept every HTTP request and/or response. Common uses: (1) Adding auth tokens to requests, (2) Global error handling (401 → redirect to login), (3) Loading spinners (start/stop on request/response), (4) Retry logic for flaky APIs, (5) Caching responses, (6) Logging for debugging. Modern Angular uses functional interceptors (`HttpInterceptorFn`) instead of class-based ones.

Q: How do you handle API errors globally?

A: Use an HTTP interceptor with `catchError`. For 401: redirect to login. For 403: show forbidden page. For 500: show generic error notification. For network errors: show offline indicator. Always `throwError` the error forward so individual components can also handle specific cases. Use a notification service to show user-friendly messages instead of raw error objects.

9. State Management

Theory

State management is how you organize, share, and update data across your application. The right approach depends on app complexity.

Approach	Complexity	When to use
Component state (signals)	Simple	Local UI state — form values, toggles, filters
Service + signals/BehaviorSubject	Medium	Shared state across a few components — cart, auth, notifications
NgRx Signal Store	Medium-High	Feature-level state with clear actions and computed state
NgRx Store (Redux pattern)	High	Large enterprise apps — actions, reducers, effects, selectors

Hands-on Code

Signal-based store (recommended for most apps)

```
// Cart store using signals – no library needed
@Injectable({ providedIn: 'root' })
export class CartStore {
  // Private writable state
  private _items = signal<CartItem[]>([]);

  // Public read-only selectors
  readonly items = this._items.asReadonly();
  readonly itemCount = computed(() => this._items().length);
  readonly total = computed(() =>
    this._items().reduce((sum, item) => sum + item.price * item.quantity, 0)
  );
  readonly isEmpty = computed(() => this._items().length === 0);

  // Actions
  addItem(product: Product) {
    this._items.update(items => {
      const existing = items.find(i => i.productId === product.id);
      if (existing) {
        return items.map(i =>
          i.productId === product.id
            ? { ...i, quantity: i.quantity + 1 }
            : i
        );
      }
      return [...items, { productId: product.id, name: product.name,
        price: product.price, quantity: 1 }];
    });
  }

  removeItem(productId: string) {
    this._items.update(items => items.filter(i => i.productId !== productId));
  }

  updateQuantity(productId: string, quantity: number) {
    if (quantity <= 0) return this.removeItem(productId);
  }
}
```



```

    this._items.update(items =>
      items.map(i => i.productId === productId ? { ...i, quantity } : i)
    );
  }

  clear() { this._items.set([]); }
}

```

Theory: This signal-based store gives you centralized state management, computed derived state (total, count, isEmpty), immutable updates, and no external library dependency. For most Angular apps with 3 years of complexity, this pattern is sufficient, scalable, and easy to test.

Interview Questions

Q: When would you introduce NgRx vs using signals + services?

A: Use signals + services when: state is shared between a few components, the team is small, and state logic is straightforward. Use NgRx when: the app has complex state transitions, you need time-travel debugging, multiple developers need strict patterns, or you need side effects orchestration (NgRx Effects). Most apps under 50 components do not need NgRx.

Q: What is the Redux pattern and how does NgRx implement it?

A: Redux pattern: single source of truth (store), state is read-only (immutable), changes via pure functions (reducers), triggered by actions. NgRx adds: Selectors (memoized state reads), Effects (side effects like API calls), and Angular DI integration. The tradeoff: more boilerplate and indirection, but very predictable state management in large teams.

10. Custom Directives & Pipes

Hands-on Code

Custom structural directive — permission-based rendering

```
// Only render content if user has the required permission
@Directive({ selector: '[appHasPermission]', standalone: true })
export class HasPermissionDirective {
  private auth = inject(AuthService);
  private templateRef = inject(TemplateRef<any>);
  private viewContainer = inject(ViewContainerRef);
  private rendered = false;

  @Input() set appHasPermission(permission: string) {
    if (this.auth.hasPermission(permission) && !this.rendered) {
      this.viewContainer.createEmbeddedView(this.templateRef);
      this.rendered = true;
    } else if (!this.auth.hasPermission(permission) && this.rendered) {
      this.viewContainer.clear();
      this.rendered = false;
    }
  }
}

// Usage:
// <button *appHasPermission="'admin:delete'">Delete User</button>
```

Custom pipe — relative time

```
@Pipe({ name: 'timeAgo', standalone: true, pure: true })
export class TimeAgoPipe implements PipeTransform {
  transform(date: Date | string): string {
    const now = Date.now();
    const time = new Date(date).getTime();
    const diff = Math.floor((now - time) / 1000);

    if (diff < 60) return 'just now';
    if (diff < 3600) return `${Math.floor(diff / 60)} minutes ago`;
    if (diff < 86400) return `${Math.floor(diff / 3600)} hours ago`;
    if (diff < 2592000) return `${Math.floor(diff / 86400)} days ago`;
    return new Date(date).toLocaleDateString();
  }
}

// Usage: {{ comment.createdAt | timeAgo }}
```

Custom attribute directive — auto-focus

```
@Directive({ selector: '[appAutoFocus]', standalone: true })
export class AutoFocusDirective implements AfterViewInit {
  private el = inject(ElementRef);

  ngAfterViewInit() {
    this.el.nativeElement.focus();
  }
}

// Usage: <input appAutoFocus />
```

Interview Questions

Q: What is the difference between structural and attribute directives?

A: Structural directives change the DOM structure — they add or remove elements (e.g., `*ngIf`, `*ngFor`, custom `*appHasPermission`). They use `TemplateRef` and `ViewContainerRef`. Attribute directives change the appearance or behavior of an existing element (e.g., `ngClass`, `ngStyle`, custom `appAutoFocus`). They use `ElementRef` and `Renderer2`.

11. Testing

Theory

Type	What it tests	Tools	Speed
Unit tests	Individual functions, services, pipes	Jest / Jasmine + Karma	Fast
Component tests	Component behavior with template	TestBed + ComponentFixture	Medium
Integration tests	Multiple components together	TestBed with routing/HTTP	Medium
E2E tests	Full app from user perspective	Cypress / Playwright	Slow

Hands-on Code

Service unit test

```
describe('CartStore', () => {
  let store: CartStore;

  beforeEach(() => {
    TestBed.configureTestingModule({});
    store = TestBed.inject(CartStore);
  });

  it('should start with empty cart', () => {
    expect(store.items()).toEqual([]);
    expect(store.isEmpty()).toBe(true);
    expect(store.total()).toBe(0);
  });

  it('should add item', () => {
    store.addItem({ id: '1', name: 'Laptop', price: 999 } as Product);
    expect(store.items().length).toBe(1);
    expect(store.total()).toBe(999);
  });

  it('should increase quantity for duplicate item', () => {
    const product = { id: '1', name: 'Laptop', price: 999 } as Product;
    store.addItem(product);
    store.addItem(product);
    expect(store.items().length).toBe(2);
    expect(store.items()[0].quantity).toBe(2);
    expect(store.total()).toBe(1998);
  });
});
```

Component test with mocked service

```
describe('ProductListComponent', () => {
  let component: ProductListComponent;
  let fixture: ComponentFixture<ProductListComponent>;
  let mockProductService: jasmine.SpyObj<ProductService>;

  beforeEach(async () => {
    mockProductService = jasmine.createSpyObj('ProductService', ['getAll']);
    mockProductService.getAll.and.returnValue(of([
      { id: '1', name: 'Laptop', price: 999 },
    ]));
  });
});
```

```

    { id: '2', name: 'Phone', price: 699 }
  ]));

  await TestBed.configureTestingModule({
    imports: [ProductListComponent],
    providers: [
      { provide: ProductService, useValue: mockProductService }
    ]
  }).compileComponents();

  fixture = TestBed.createComponent(ProductListComponent);
  component = fixture.componentInstance;
  fixture.detectChanges();
});

it('should display products', () => {
  const items = fixture.nativeElement.querySelectorAll('.product-item');
  expect(items.length).toBe(2);
  expect(items[0].textContent).toContain('Laptop');
});

it('should call service on init', () => {
  expect(mockProductService.getAll).toHaveBeenCalledTimes(1);
});
});

```

Interview Questions

Q: How do you test a component that depends on an HTTP service?

A: Create a mock/spy of the service and provide it in TestBed. Use `jasmine.createSpyObj` or a manual mock class. The mock returns controlled data (using `of()` for observables). This isolates the component from real HTTP calls and lets you test behavior for various data scenarios, errors, and edge cases without a backend.

Q: What is the testing pyramid and how does it apply to Angular?

A: Many unit tests (fast, cheap), fewer integration tests (test components together), very few E2E tests (slow, brittle). In Angular: unit test services, pipes, and pure logic heavily. Component tests for template behavior and user interactions. E2E tests (Cypress/Playwright) for critical user journeys only — login, checkout, onboarding.

12. Angular Security

Theory

Angular has built-in protections against common web vulnerabilities, but you must understand them to use them correctly.

Key security topics

- **XSS (Cross-Site Scripting):** Angular auto-sanitizes template bindings. Interpolation (`{{ }}`) is always safe. `[innerHTML]` is sanitized. `bypassSecurityTrustHtml` skips sanitization — use with extreme caution.
- **CSRF (Cross-Site Request Forgery):** Use Angular's `HttpClientXsrfModule` which reads a token from a cookie and sends it as a header. The server must set and validate this token.
- **Authentication:** Use JWT tokens stored in `HttpOnly` cookies (not `localStorage`). Add auth interceptor to attach tokens. Handle token refresh with interceptors.
- **Authorization:** Use route guards (`canActivate`) for page-level access. Use structural directives (`*appHasPermission`) for element-level access. Always enforce authorization on the server — client-side guards are UX, not security.
- **Content Security Policy (CSP):** Set CSP headers to prevent inline scripts, restrict allowed origins, and block unsafe patterns.
- **Dependency vulnerabilities:** Run `npm audit` regularly. Use tools like Snyk or Dependabot for automated vulnerability scanning.

Hands-on Code

JWT refresh interceptor

```
export const tokenRefreshInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);

  return next(req).pipe(
    catchError((error: HttpResponse) => {
      if (error.status === 401 && !req.url.includes('/refresh')) {
        return auth.refreshToken().pipe(
          switchMap(newToken => {
            const cloned = req.clone({
              setHeaders: { Authorization: `Bearer ${newToken}` }
            });
            return next(cloned);
          }),
          catchError(() => {
            auth.logout();
            return throwError(() => error);
          })
        );
      }
      return throwError(() => error);
    })
  );
};
```

Interview Questions

Q: How does Angular protect against XSS?

A: Angular treats all template values as untrusted by default. Interpolation (`{{ }}`) escapes HTML entities. Property binding (`[innerHTML]`) is sanitized — dangerous tags and attributes are stripped. Only `DomSanitizer.bypassSecurityTrust*` disables sanitization, and it should be used only for trusted content with additional server-side validation.

Q: Where should you store JWT tokens?

A: HttpOnly cookies are the most secure — they are not accessible via JavaScript, preventing XSS-based token theft. If you must use localStorage/sessionStorage (e.g., for third-party auth), understand the XSS risk and implement strict CSP. Never store tokens in plain JavaScript variables that persist in memory longer than needed.

13. Angular CLI, Build & Deployment

Theory

- **ng build --configuration production:** AOT compilation, tree shaking, minification, dead-code elimination, source map control.
- **AOT vs JIT:** AOT (Ahead of Time) compiles templates at build time — faster startup, smaller bundles, earlier error detection. JIT (Just in Time) compiles at runtime — used only for development.
- **Environments:** environment.ts / environment.prod.ts for configuration. Use injection tokens for runtime config.
- **Bundle analysis:** `ng build --stats-json + webpack-bundle-analyzer` to visualize chunk sizes.
- **Differential loading:** Angular can produce modern ES2020+ bundles for modern browsers and ES5 fallbacks for legacy.
- **SSR with Angular Universal / @angular/ssr:** Server-side rendering for SEO and faster perceived loading.
- **Docker:** Multi-stage builds — build in Node container, serve from nginx container.

Hands-on Code

Dockerfile for Angular production

```
# Stage 1: Build
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npx ng build --configuration production

# Stage 2: Serve
FROM nginx:alpine
COPY --from=build /app/dist/my-app/browser /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

nginx.conf for Angular SPA

```
server {
    listen 80;

    location / {
        root /usr/share/nginx/html;
        try_files $uri $uri/ /index.html; # SPA fallback
    }

    # Cache static assets aggressively
    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff2)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
}
```

Interview Questions

Q: What is AOT compilation and why is it important?

A: AOT compiles Angular templates and components into efficient JavaScript during the build step, not at runtime. Benefits: (1) Faster startup — no compilation in the browser. (2) Smaller bundles — the compiler is not shipped to the browser. (3) Earlier error detection — template errors are caught at build time. (4) Better security — templates are not shipped as strings. Angular uses AOT by default since Angular 9.

Q: How do you optimize Angular bundle size?

A: (1) Lazy load routes. (2) Use standalone components for tree-shaking. (3) Remove unused imports and dead code. (4) Use `@defer` for below-fold content. (5) Set bundle budgets in `angular.json`. (6) Analyze bundles with `webpack-bundle-analyzer`. (7) Use production builds with optimization enabled. (8) Avoid importing entire libraries — use subpath imports.

14. Styling & CSS Architecture in Angular

Theory

- **View Encapsulation:** Angular scopes component styles by default (Emulated mode — adds unique attributes). `ViewEncapsulation.None` makes styles global. `ViewEncapsulation.ShadowDom` uses native Shadow DOM.
- **::ng-deep:** Pierces encapsulation to style child components. Deprecated but still used — prefer global styles or CSS variables instead.
- **CSS Variables:** Best way to create themeable components across encapsulation boundaries.
- **SCSS:** Angular CLI supports SCSS out of the box. Use variables, mixins, and nesting for maintainable styles.
- **Responsive design:** CSS Grid, Flexbox, media queries, container queries (modern).

Interview Questions

Q: What is Angular view encapsulation?

A: Angular adds unique attributes to component elements and scopes CSS selectors to those attributes. This prevents styles from leaking between components. Three modes: `Emulated` (default — uses attribute selectors), `ShadowDom` (uses native Shadow DOM), and `None` (global styles). Emulated is the most commonly used because it works in all browsers without the limitations of Shadow DOM.

15. Design Patterns & Best Practices

Theory

15.1 SOLID principles in Angular

Principle	Angular application
Single Responsibility	Components handle UI, services handle business logic, pipes handle transformation
Open/Closed	Components accept @Input to customize behavior without modification
Liskov Substitution	Services implement interfaces; mocks in tests follow the same contract
Interface Segregation	Small, focused interfaces instead of god-services. Many small services > one giant service
Dependency Inversion	Depend on InjectionToken abstractions, not concrete implementations

15.2 Angular-specific patterns

- **Smart/Dumb components:** Containers fetch data, presentational components display it.
- **Facade pattern:** A service that simplifies complex subsystem interactions into a clean API.
- **Adapter pattern:** A service that transforms API data into the shape your components need.
- **Strategy pattern:** Use DI to inject different implementations based on context (e.g., different payment processors).
- **Repository pattern:** Abstract data access behind a service interface — swap between REST, GraphQL, or local storage.
- **Feature modules / feature folders:** Organize code by feature, not by type. Each feature has its own components, services, routes, and models.

15.3 File structure for scalable Angular apps

```
src/
├── app/
│   ├── core/           # Singleton services, guards, interceptors
│   │   ├── auth/
│   │   ├── http/
│   │   ├── guards/
│   │   ├── shared/     # Reusable components, directives, pipes
│   │   │   ├── components/
│   │   │   ├── directives/
│   │   │   └── pipes/
│   │   └── features/   # Feature modules/routes
│   │       ├── dashboard/
│   │       ├── products/
│   │       ├── orders/
│   │       └── admin/
│   ├── models/         # TypeScript interfaces and types
│   └── app.routes.ts
├── assets/
├── environments/
└── styles/
```

Interview Questions

Q: How do you structure a large Angular application?

A: Feature-based folder structure: core (singletons — auth, HTTP, guards), shared (reusable UI — buttons, pipes, directives), features (self-contained feature areas with their own components, services, and routes). Each feature is lazy-loaded. Core services are `providedIn: 'root'`. Models are shared across features via a models directory. This scales because each feature is independent and can be developed, tested, and deployed separately.

Q: What is the facade pattern in Angular?

A: A facade service provides a simplified API over complex subsystems. Example: a `CheckoutFacade` that coordinates `CartService`, `PaymentService`, `ShippingService`, and `OrderService`. The component only interacts with the facade, not the individual services. Benefits: components stay simple, business logic is centralized, and the underlying services can be refactored without changing components.

16. Performance Optimization

Performance is covered in extensive detail in the companion document: **Angular and Frontend Performance Optimization Guide** (28 sections, 60+ code examples, hands-on interview Q&A). Key topics covered there:

- Change detection optimization (OnPush, signals, zone pollution)
- Bundle optimization (lazy loading, @defer, code splitting, budgets)
- Core Web Vitals (LCP, INP, CLS) — theory, measurement, and fixes
- Rendering pipeline, long tasks, layout thrashing
- RxJS performance patterns (switchMap, shareReplay, debounce)
- Memory leak detection and prevention
- Production RUM monitoring setup
- Performance tooling (Lighthouse, Chrome DevTools, Angular DevTools)
- SSR, hydration, and hybrid rendering
- 50+ interview questions with strong answers

Refer to: `angular-frontend-performance-guide.html` in the `performance/` directory for the complete performance deep dive.

17. Angular Signals — Complete Guide

Theory

Signals are Angular's fine-grained reactive primitive introduced in Angular 16 and production-ready in Angular 17+. They represent a synchronous, auto-tracking reactive value that notifies consumers when it changes.

17.1 Signal API

API	Purpose	Example
<code>signal(value)</code>	Create a writable signal	<code>count = signal(0)</code>
<code>signal.set(newValue)</code>	Replace the value	<code>count.set(5)</code>
<code>signal.update(fn)</code>	Update based on current value	<code>count.update(c => c + 1)</code>
<code>computed(() => ...)</code>	Derived value, auto-tracks dependencies	<code>double = computed(() => count() * 2)</code>
<code>effect(() => ...)</code>	Side effect that runs when signals it reads change	Logging, local storage sync
<code>input()</code>	Signal-based component input	<code>name = input<string>()</code>
<code>input.required()</code>	Required signal input	<code>id = input.required<number>()</code>
<code>output()</code>	Signal-based component output	<code>clicked = output<void>()</code>
<code>model()</code>	Two-way binding signal	<code>value = model('')</code>
<code>toSignal(observable)</code>	Convert Observable to Signal	<code>user = toSignal(user\$)</code>
<code>toObservable(signal)</code>	Convert Signal to Observable	<code>count\$ = toObservable(count)</code>

Hands-on Code

Complete signal-based component

```
@Component({
  selector: 'app-todo-list',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <h2>Todos ({ { remaining() } } remaining)</h2>

    <input #newTodo (keyup.enter)="addTodo(newTodo.value); newTodo.value = ''" />

    <select (change)="filter.set($any($event.target).value)">
      <option value="all">All</option>
      <option value="active">Active</option>
      <option value="completed">Completed</option>
    </select>

    @for (todo of filteredTodos(); track todo.id) {
      <div [class.completed]="todo.done">
        <input type="checkbox" [checked]="todo.done"
          (change)="toggleTodo(todo.id)" />
        {{ todo.text }}
        <button (click)="removeTodo(todo.id)">x</button>
      </div>
    }
  `
})
```

```

    },
  },
});
export class TodoListComponent {
  // State
  todos = signal<Todo[]>([]);
  filter = signal<'all' | 'active' | 'completed'>('all');

  // Derived state
  remaining = computed(() => this.todos().filter(t => !t.done).length);
  filteredTodos = computed(() => {
    const all = this.todos();
    switch (this.filter()) {
      case 'active': return all.filter(t => !t.done);
      case 'completed': return all.filter(t => t.done);
      default: return all;
    }
  });

  // Side effect: persist to localStorage
  constructor() {
    // Load from localStorage
    const saved = localStorage.getItem('todos');
    if (saved) this.todos.set(JSON.parse(saved));

    // Save whenever todos change
    effect(() => {
      localStorage.setItem('todos', JSON.stringify(this.todos()));
    });
  }

  addTodo(text: string) {
    if (!text.trim()) return;
    this.todos.update(list => [...list, { id: crypto.randomUUID(), text: text.trim(), done: false }]);
  }

  toggleTodo(id: string) {
    this.todos.update(list => list.map(t => t.id === id ? { ...t, done: !t.done } : t));
  }

  removeTodo(id: string) {
    this.todos.update(list => list.filter(t => t.id !== id));
  }
}

```

Interview Questions

Q: How do signals change Angular's reactivity model?

A: Traditional Angular uses Zone.js to detect async events and run change detection on the entire component tree. Signals create a fine-grained dependency graph — when a signal changes, only the specific computed values and effects that depend on it are recalculated. This is more precise (less wasted work) and enables zoneless Angular where CD is driven entirely by signal changes, not by async event interception.

Q: When would you use effect() and what are the gotchas?

A: Use `effect()` for side effects: localStorage sync, analytics logging, console debugging, DOM manipulation. Gotchas: (1) Effects run at least once on creation. (2) Don't write to signals inside effects (creates dependency loops) — use `untracked()` to read without tracking. (3) Effects are cleaned up when the component is destroyed. (4) Effects are not for derived state — use `computed()` for that.

Video Resources

- [Angular Signals Explained — Joshua Morony](#)
- [Angular Signals Tutorial in Hindi](#)

- [Angular Official Signals Guide](#)

18. Modern Angular Features (v17+)

18.1 New control flow syntax

<pre>// Old syntax *ngIf="condition" *ngFor="let item of items" [ngSwitch]="value"</pre>	<pre>// New syntax (Angular 17+) @if (condition) { ... } @else { ... } @for (item of items; track item.id) { ... } @empty { .. @switch (value) { @case ('a') { ... } @default { ... }</pre>
--	---

Theory: New control flow is built into the compiler (not directives), resulting in smaller bundles and better performance. `@for` requires `track` — preventing the common mistake of forgetting `trackBy`. `@empty` provides built-in empty state handling.

18.2 Deferrable views

```
@defer (on viewport; prefetch on idle) {
  <app-heavy-chart [data]="chartData()" />
} @placeholder {
  <div class="skeleton" style="height:300px">Chart loading...</div>
} @loading (after 100ms; minimum 500ms) {
  <div class="spinner">Loading chart...</div>
} @error {
  <p>Failed to load chart.</p>
}
```

Triggers: on viewport, on idle, on interaction, on hover, on timer(5s), on immediate, when condition. Prefetch triggers can be different from render triggers.

18.3 Other modern features

- **Standalone by default:** No NgModules needed for new projects.
- **Functional guards & interceptors:** Plain functions instead of classes.
- **inject() function:** Use DI outside constructors — in field initializers, factory functions.
- **Required inputs:** `@Input({ required: true })` or `input.required()`.
- **Input transforms:** `@Input({ transform: booleanAttribute })` for automatic coercion.
- **Router input binding:** Automatically bind route params to component inputs.
- **SSR hydration:** Full and incremental hydration for server-rendered apps.

Interview Questions

Q: What are the biggest changes in Angular 17?

A: (1) New built-in control flow (`@if`, `@for`, `@switch`) replacing structural directives. (2) Deferrable views (`@defer`) for lazy loading template parts. (3) Standalone components as default. (4) Signals stabilized for production use. (5) ESBuild-based build system (vite dev server) for faster builds. (6) New Angular.dev documentation site. These changes make Angular simpler, faster, and more modern.

19. Frontend System Design

Theory

Senior interviews often include a system design round. For frontend developers, this means designing the architecture of a client-side application — not backend infrastructure.

Common frontend system design questions

- Design an e-commerce product listing page with filters, sorting, and pagination
- Design a real-time chat application
- Design a dashboard with multiple widgets and real-time data
- Design a form builder that generates dynamic forms
- Design an infinite scroll feed (social media style)
- Design a notification system

Framework for answering

1. **Clarify requirements:** functional (what it does), non-functional (performance, scale, accessibility)
2. **Component architecture:** break the UI into components, define the tree structure
3. **Data flow:** what state exists, where it lives, how components communicate
4. **API design:** what endpoints are needed, request/response shapes, caching
5. **Performance considerations:** lazy loading, virtualization, pagination, caching
6. **Error handling:** network errors, empty states, loading states, retry logic
7. **Accessibility:** keyboard navigation, screen readers, ARIA attributes
8. **Testing strategy:** what to unit test, what to E2E test

Hands-on: Design walk-through

Example: E-commerce product listing

```
// Component tree
AppComponent
  ■■■ HeaderComponent (search bar, cart icon with count)
  ■■■ ProductListingPage
    ■ ■■■ FilterSidebar (category, price range, brand)
    ■ ■■■ SortDropdown (price asc/desc, rating, newest)
    ■ ■■■ ProductGrid
      ■ ■■■ ProductCard (image, name, price, rating, add-to-cart)
    ■ ■■■ Pagination
  ■■■ CartSidebar (items, total, checkout button)

// State management
- FilterStore: signals for active filters, sort, page
- ProductStore: signal for products, loading, error, total count
- CartStore: signal for cart items, computed total

// API
GET /api/products?category=X&priceMin=Y&priceMax=Z&sort=price_asc&page=1&limit=20
Response: { items: Product[], total: number, page: number }

// Performance
- Virtual scroll for large product grids
- Image lazy loading with placeholder
- Debounced filter changes (don't API-call on every slider tick)
```


- @defer for filter sidebar on mobile
- CDN for product images with responsive srcset

Interview Questions

Q: How would you design an infinite scroll feed?

A: (1) Use `IntersectionObserver` on a sentinel element at the bottom. (2) When visible, fetch next page from API. (3) Append results to the signal array. (4) Use `@for . . track` with stable IDs. (5) For performance: virtual scroll if the list can grow to thousands of items. (6) Handle: loading state, error retry, empty state, end-of-content indicator. (7) Cache fetched pages in a service so back-navigation doesn't re-fetch.

20. Accessibility (a11y)

Key topics

- **Semantic HTML:** Use proper elements — `<button>` not `<div onclick>`, `<nav>`, `<main>`, `<section>`, headings in order.
- **ARIA attributes:** `aria-label`, `aria-expanded`, `aria-live`, `role` — use when semantic HTML is not enough.
- **Keyboard navigation:** All interactive elements must be keyboard-accessible. Test with Tab, Enter, Escape.
- **Focus management:** After navigation or modal open, move focus appropriately. Use `cdkTrapFocus` for dialogs.
- **Color contrast:** WCAG AA requires 4.5:1 for text, 3:1 for large text.
- **Angular CDK a11y module:** `FocusTrap`, `LiveAnnouncer`, `FocusMonitor`.
- **Testing:** Use `axe-core` or `Lighthouse` accessibility audit. Test with screen reader (NVDA, VoiceOver).

Interview Questions

Q: How do you make an Angular dialog accessible?

A: (1) Use `role="dialog"` and `aria-modal="true"`. (2) Set `aria-labelledby` pointing to the dialog title. (3) Trap focus inside the dialog using `cdkTrapFocus`. (4) Return focus to the trigger element when dialog closes. (5) Close on Escape key press. (6) Prevent scrolling of background content. Angular CDK Dialog handles most of this automatically.

21. Complete Interview Preparation Checklist

Technical topics by priority

Priority	Topic	Expected depth at 3 years
■ Must know	Components, lifecycle hooks, DI	Deep — explain internals, edge cases
■ Must know	Change detection (Default, OnPush, Signals)	Deep — explain mechanism, optimize
■ Must know	RxJS (operators, subjects, error handling)	Deep — write complex pipelines
■ Must know	Routing (lazy loading, guards, resolvers)	Deep — complex route configurations
■ Must know	Reactive forms (validation, dynamic, typed)	Deep — build complex forms from scratch
■ Must know	HTTP (interceptors, error handling)	Deep — write interceptors from scratch
■ Should know	State management (signals, NgRx basics)	Can design a signal-based store
■ Should know	Testing (unit + component)	Can write tests with mocked services
■ Should know	Performance optimization	Can explain and apply key optimizations
■ Should know	Security (XSS, CSRF, auth patterns)	Can explain threats and mitigations
■ Should know	Modern Angular (17+ features)	Using @if/@for/@defer, signals, standalone
■ Good to know	SSR / hydration	Can explain when and why
■ Good to know	Accessibility	Basic ARIA, keyboard nav, semantic HTML
■ Good to know	System design	Can break down a frontend system
■ Good to know	CI/CD, Docker, deployment	Basic production setup

Behavioral / soft skill questions to prepare

1. Tell me about a challenging bug you debugged and how you solved it.
2. Describe a time you optimized performance in an Angular app.
3. How do you handle disagreements in code reviews?
4. How do you decide between building a feature vs using a library?
5. How do you mentor junior developers?
6. Describe your process for learning new Angular features.

7. How do you balance code quality with delivery speed?

Practice plan

Week	Focus	Action
Week 1	JS/TS fundamentals + Angular core	Review closures, promises, event loop. Build a component with lifecycle hooks and DI.
Week 2	RxJS + Change detection	Build search autocomplete from scratch. Profile CD with Angular DevTools.
Week 3	Forms + HTTP + Routing	Build a multi-step form with validation, guards, and API integration.
Week 4	State management + Testing	Build a todo/cart app with signal store. Write unit and component tests.
Week 5	Performance + Modern features	Study performance guide. Use @defer, signals, new control flow in a demo app.
Week 6	Mock interviews + System design	Practice explaining concepts aloud. Do 2-3 system design exercises.

22. Complete Video Resource List

Angular Complete Courses

- [Angular Complete Guide](#) — Maximilian Schwarzmüller
- [Angular Tutorial for Beginners](#) — Codevolution
- [Angular Full Course in Hindi](#) — Code Step By Step
- [Angular Official Tutorial](#) (angular.dev)

TypeScript & JavaScript

- [Namaste JavaScript by Akshay Saini \[Hindi\]](#)
- [TypeScript Tutorial in Hindi](#)
- [What the heck is the event loop?](#) — Philip Roberts

RxJS

- [RxJS Top Ten Operators](#) — Fireship
- [RxJS Complete Course](#) — Academind
- [RxJS Official Documentation](#)

Angular Signals

- [Angular Signals Explained](#) — Joshua Morony
- [Angular Signals Tutorial in Hindi](#)
- [Angular Official Signals Guide](#)

Performance

- [Angular Performance Optimization Strategy](#)
- [Profiling Angular with Chrome DevTools](#)
- [Core Web Vitals Workshop](#)
- [Core Web Vitals full course in Hindi](#)

Angular Interview Preparation

- [Top 50 Angular Interview Questions](#)
- [Angular Interview Questions 2024](#) (search)

Official Documentation (most important)

- [Angular Official Docs](#) (angular.dev)
- [Signals Guide](#)
- [Deferrable Views Guide](#)
- [Runtime Performance Guide](#)
- [TypeScript Documentation](#)
- [MDN JavaScript Reference](#)

23. Micro-Frontend Architecture

Theory

Micro-frontends extend the microservice idea to the frontend: independently deployable UI applications composed into a single user experience. At the SDE-3 level you are expected to evaluate whether your org needs micro-frontends, choose an integration strategy, and own the cross-cutting concerns (shared state, routing, design system, auth).

23.1 Integration strategies

Strategy	How it works	Pros	Cons
Module Federation (Webpack 5 / Vite)	Remotes expose modules at build time; host loads them at runtime over HTTP	Fine-grained sharing (components, services), independent deployments, shared deps	Webpack/Vite coupling, version-compat risk, complex build config
Web Components / Custom Elements	Each micro-app is bundled as a custom element; host renders <code><mfe-checkout></code>	Framework-agnostic, true encapsulation, simple host contract	Shadow DOM styling challenges, larger per-app bundles, limited framework interop
Native Federation (Angular-native)	@angular-architects/native-federation — ESM-based approach without Webpack	Works with Angular CLI esbuild, no Webpack needed, tree-shakeable remotes	Relatively new, community still maturing
iframes	Each micro-app runs in an isolated iframe	Total isolation, crash containment	Poor UX (scrolling, resizing), no shared state, accessibility issues
Route-based (Nx + lazy routes)	Monorepo with each feature as a lazy-loaded library; single build artefact	Simple, full type safety, shared DI tree	Not independently deployable — it's a modular monolith

23.2 Cross-cutting concerns

- **Shared shell:** A host app that provides the layout, header, nav, auth context, and router. Remotes plug into named router outlets or designated DOM slots.
- **Shared dependencies:** Module Federation's `shared` config prevents loading Angular/RxJS/CDK multiple times. Pin major versions across teams. Use a `singleton: true, strictVersion: true` policy for core deps.
- **Communication:** Custom Events on `window` for loose coupling. Shared signal-based services for tighter coupling within the same runtime. Never share state via global variables.
- **Routing:** Shell owns top-level routes. Each remote defines child routes. Use `loadRemoteModule()` in the route config. Deep-link support requires coordination on URL namespace.
- **Auth:** Shell handles login and stores the token. Remotes retrieve it via a shared auth service or custom event. Never duplicate auth logic across remotes.
- **Design consistency:** Publish a shared design-system library (CSS variables, Angular CDK components) consumed by all remotes.

Hands-on Code

Module Federation — host route config

```
// host app — app.routes.ts
export const routes: Routes = [
  { path: '', component: ShellComponent, children: [
    { path: 'dashboard', loadComponent: () =>
      import('./dashboard/dashboard.component').then(m => m.DashboardComponent) },
    { path: 'checkout', loadRemoteModule({
      type: 'module',
      remoteEntry: 'https://checkout.cdn.example.com/remoteEntry.js',
      exposedModule: './CheckoutRoutes'
    }).then(m => m.CHECKOUT_ROUTES)
    },
    { path: 'inventory', loadRemoteModule({
      type: 'module',
      remoteEntry: 'https://inventory.cdn.example.com/remoteEntry.js',
      exposedModule: './InventoryRoutes'
    }).then(m => m.INVENTORY_ROUTES)
    }
  ]
}
];
```

Module Federation — webpack shared config

```
// webpack.config.ts (host)
shared: share({
  '@angular/core': { singleton: true, strictVersion: true, requiredVersion: '^17.0.0' },
  '@angular/common': { singleton: true, strictVersion: true, requiredVersion: '^17.0.0' },
  '@angular/router': { singleton: true, strictVersion: true, requiredVersion: '^17.0.0' },
  'rxjs': { singleton: true, strictVersion: true, requiredVersion: '^7.8.0' },
  '@angular/common/http': { singleton: true, strictVersion: true, requiredVersion: '^17.0.0' },
})
```

Cross-micro-frontend communication via Custom Events

```
// In remote A — dispatch event
window.dispatchEvent(new CustomEvent('mfe:cart-updated', {
  detail: { itemCount: 5, total: 2499 }
}));

// In remote B or shell — listen
@Component({ ... })
export class HeaderComponent implements OnInit, OnDestroy {
  cartCount = signal(0);
  private handler = (e: Event) => {
    const detail = (e as CustomEvent).detail;
    this.cartCount.set(detail.itemCount);
  };

  ngOnInit() {
    window.addEventListener('mfe:cart-updated', this.handler);
  }

  ngOnDestroy() {
    window.removeEventListener('mfe:cart-updated', this.handler);
  }
}
```

Interview Questions

Q: When would you recommend micro-frontends vs a modular monolith?

A: Micro-frontends when: (1) multiple autonomous teams own different business domains, (2) independent deployment is a hard requirement, (3) the app is large enough that a single build takes >10 min, (4) different domains have different release cadences. Modular monolith (Nx libraries + lazy loading) when: one or two teams, shared release cycle, full type-safety is more valuable than deployment independence. In most orgs under ~50 frontend engineers, the modular monolith is simpler and faster.

Q: How do you handle version mismatches between shared Angular in micro-frontends?

A: Pin the major Angular version across all remotes using `singleton: true, strictVersion: true` in Module Federation's shared config. If versions mismatch, the runtime throws instead of silently loading two Angulars (which would crash). Use a monorepo or a shared package.json policy enforced in CI to keep versions aligned. Minor and patch divergence is usually safe.

Q: What are the biggest pitfalls of micro-frontends?

A: (1) Duplicate bundles if shared config is wrong — loading Angular twice is ~80KB wasted. (2) Inconsistent UX if teams don't share a design system. (3) Communication complexity — global state leaks, event naming collisions. (4) Harder E2E testing — you need to deploy all remotes to test a flow. (5) DevEx regression — local dev requires running multiple apps or using a mock shell. Evaluate the operational cost vs the autonomy benefit carefully.

Video Resources

- [Module Federation with Angular — Manfred Steyer](#)
- [Micro Frontends Explained — Jack Herrington](#)
- [Angular Architects — Module Federation Guide](#)

24. Monorepo Tooling with Nx

Theory

Nx is the de-facto monorepo tool for enterprise Angular projects. At the SDE-3 level you own the workspace architecture — library boundaries, dependency constraints, caching strategy, and CI pipelines.

24.1 Core Nx concepts

Concept	What it does	SDE-3 responsibility
Workspace	A single repo containing multiple apps and libs	Design library boundaries, enforce module types
Libraries	feature, ui, data-access, util — standard types	Define lib taxonomy, publish conventions
Dependency graph	<code>nx graph</code> visualizes inter-lib dependencies	Review for cycles, enforce layering
Affected commands	<code>nx affected:test</code> only runs tests for changed libs	CI speed optimization — test only what changed
Computation caching	Local and remote (Nx Cloud) task caching	Configure Nx Cloud, set cache TTL, define cacheable targets
Module boundary rules	<code>@nx/enforce-module-boundaries</code> ESLint rule	Tag libs (scope:checkout, type:feature), define allowed dependencies
Generators & Executors	Code scaffolding and custom build pipelines	Create org-specific generators for consistent project creation

24.2 Library taxonomy

```
libs/
├── shared/
│   ├── ui/           # Reusable presentational components (buttons, cards, dialogs)
│   ├── util/         # Pure utility functions (date formatters, validators)
│   ├── data-access/  # API services, HTTP clients, interceptors
│   └── models/       # TypeScript interfaces shared across domains
├── checkout/
│   ├── feature-cart/ # Smart cart component, cart page
│   ├── feature-payment/ # Payment form, payment confirmation
│   ├── data-access/  # Checkout API service
│   └── ui/           # Checkout-specific presentational components
├── inventory/
│   ├── feature-list/
│   ├── feature-detail/
│   └── data-access/
├── auth/
│   ├── feature-login/
│   ├── feature-register/
│   └── data-access/
```

Hands-on Code

Module boundary rules — `.eslintrc.json`

```
{
  "@nx/enforce-module-boundaries": [
    "error",
```

```

    {
      "enforceBuildableLibDependency": true,
      "allow": [],
      "depConstraints": [
        { "sourceTag": "type:feature", "onlyDependOnLibsWithTags": ["type:data-access", "type:ui"] },
        { "sourceTag": "type:data-access", "onlyDependOnLibsWithTags": ["type:util", "type:models"] },
        { "sourceTag": "type:ui", "onlyDependOnLibsWithTags": ["type:util", "type:models"] },
        { "sourceTag": "type:util", "onlyDependOnLibsWithTags": ["type:models"] },
        { "sourceTag": "scope:checkout", "onlyDependOnLibsWithTags": ["scope:checkout", "scope:inventory"] },
        { "sourceTag": "scope:inventory", "onlyDependOnLibsWithTags": ["scope:inventory", "scope:checkout"] }
      ]
    }
  ]
}

```

Theory: This prevents checkout code from importing inventory code directly. Cross-domain communication must go through a shared service or event system. This enforces domain boundaries at build time — a violation fails the lint step in CI.

Nx affected in CI

```

# .github/workflows/ci.yml
jobs:
  main:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - uses: nrwl/nx-set-shas@v4 # Determines base SHA for affected
      - run: npm ci
      - run: npx nx affected -t lint test build --parallel=3
      # Only lint, test, build libraries that changed or are affected by changes

```

Interview Questions

Q: How do you decide library boundaries in an Nx monorepo?

A: Slice by domain first (checkout, inventory, auth), then by type (feature, data-access, ui, util). A feature lib is a smart component that wires up data-access and ui libs. Data-access libs contain services and API calls. UI libs are pure presentational components. Utils are pure functions with no Angular dependency. Tag every lib with scope and type, then enforce boundaries with the ESLint rule. This prevents spaghetti dependencies and makes affected calculations precise.

Q: What is Nx computation caching and how does it speed up CI?

A: Nx hashes each task's inputs (source files, dependencies, env variables, config). If the hash matches a previous run, it replays the cached output instead of re-executing. Locally this avoids redundant rebuilds. With Nx Cloud, the cache is shared across CI agents and developers — if one developer builds a lib, another developer or CI gets the cached result instantly. This can reduce CI time by 50-80% in large monorepos.

25. Advanced Dependency Injection Patterns

Theory

At the SDE-3 level, you need to go beyond basic `providedIn: 'root'`. You must understand multi-providers, injection context, `forwardRef`, tree-shakeable tokens, and how the injector hierarchy is used to scope services precisely.

Hands-on Code

InjectionToken for configuration

```
// Define a type-safe injection token
export interface AppConfig {
  apiBaseUrl: string;
  featureFlags: Record<string, boolean>;
  maxRetries: number;
  environment: 'development' | 'staging' | 'production';
}

export const APP_CONFIG = new InjectionToken<AppConfig>('AppConfig');

// Provide in app config
export const appConfig: ApplicationConfig = {
  providers: [
    {
      provide: APP_CONFIG,
      useValue: {
        apiBaseUrl: 'https://api.example.com',
        featureFlags: { darkMode: true, betaCheckout: false },
        maxRetries: 3,
        environment: 'production'
      } satisfies AppConfig
    }
  ]
};

// Consume anywhere
@Injectable({ providedIn: 'root' })
export class ApiService {
  private config = inject(APP_CONFIG);
  private baseUrl = this.config.apiBaseUrl;
}
```

Multi-providers — plugin architecture

```
// Define a token that can have MULTIPLE providers
export const VALIDATOR = new InjectionToken<FormValidator>('FormValidator', {
  factory: () => [], // default empty
});

export interface FormValidator {
  name: string;
  validate(value: unknown): string | null;
}

// Register multiple validators
export const appConfig: ApplicationConfig = {
  providers: [
    { provide: VALIDATOR, useClass: RequiredValidator, multi: true },
    { provide: VALIDATOR, useClass: EmailValidator, multi: true },
    { provide: VALIDATOR, useClass: PasswordStrengthValidator, multi: true },
  ]
};

// Consume – you receive an array
```

```

@Injectable({ providedIn: 'root' })
export class ValidationService {
  private validators = inject(VALIDATOR) as FormValidator[];

  validate(value: unknown): string[] {
    return this.validators
      .map(v => v.validate(value))
      .filter((msg): msg is string => msg !== null);
  }
}
// Theory: Multi-providers are how Angular's own HTTP_INTERCEPTORS, ENVIRONMENT_INITIALIZER,
// and APP_INITIALIZER work. They enable a plugin architecture where consumers
// don't know about specific implementations.

```

forwardRef — resolving circular dependencies

```

// When two classes reference each other in their decorators
@Component({
  selector: 'app-tree-node',
  template: `
    <span>{{ node.label }}</span>
    @for (child of node.children; track child.id) {
      <app-tree-node [node]="child" /> <!-- Self-reference! -->
    }
  `,
  imports: [forwardRef(() => TreeNodeComponent)] // Resolve circular ref
})
export class TreeNodeComponent {
  node = input.required<TreeNode>();
}

```

Component-level DI — scoped state

```

// A service that holds form state – each form instance gets its own copy
@Injectable() // NOT providedIn: 'root'
export class FormEditorState {
  private _dirty = signal(false);
  private _errors = signal<Record<string, string>>({});

  readonly dirty = this._dirty.asReadOnly();
  readonly errors = this._errors.asReadOnly();
  readonly hasErrors = computed(() => Object.keys(this._errors()).length > 0);

  markDirty() { this._dirty.set(true); }
  setError(field: string, msg: string) {
    this._errors.update(e => ({ ...e, [field]: msg }));
  }
  clearError(field: string) {
    this._errors.update(e => {
      const { [field]: _, ...rest } = e;
      return rest;
    });
  }
  reset() { this._dirty.set(false); this._errors.set({}); }
}

// Each editor component gets its OWN instance
@Component({
  selector: 'app-user-editor',
  providers: [FormEditorState], // component-scoped
  template: `
    @if (formState.hasErrors()) {
      <div class="error-banner">Please fix errors before saving.</div>
    }
    ...
  `
})

```

```

export class UserEditorComponent {
  protected formState = inject(FormEditorState);
}

// Another editor on the same page gets a DIFFERENT instance
@Component({
  selector: 'app-address-editor',
  providers: [FormEditorState],
  template: `...`
})
export class AddressEditorComponent {
  protected formState = inject(FormEditorState);
}

```

inject() context — factory and functional patterns

```

// inject() works in field initializers and factory functions
// Not just in constructors!

// Factory function using inject context
function createLogger(prefix: string) {
  const http = inject(HttpClient);
  const config = inject(APP_CONFIG);

  return {
    log: (msg: string) => {
      console.log(`[${prefix}] ${msg}`);
      if (config.environment === 'production') {
        http.post('/api/logs', { prefix, msg, ts: Date.now() }).subscribe();
      }
    }
  };
}

// Usage in a component field initializer
@Component({ ... })
export class DashboardComponent {
  private logger = createLogger('Dashboard');
  // Works because the component constructor's injection context
  // is active during field initialization
}

```

Interview Questions

Q: What is the difference between useClass, useValue, useFactory, and useExisting?

A: `useClass`: creates a new instance of the class. `useValue`: provides a static value (configs, constants).

`useFactory`: calls a factory function that can use `inject()` to read other providers — flexible for conditional logic. `useExisting`: creates an alias to another token — useful when a class implements multiple interfaces and you want the same singleton accessible via different tokens.

Q: What is the injection context and when does inject() work?

A: `inject()` works in: constructors of classes created by DI, field initializers of such classes, and factory functions passed to `useFactory`, `InjectionToken`, `runInInjectionContext()`. It does NOT work in: lifecycle hooks (`ngOnInit`, etc.), event handlers, `setTimeout` callbacks, or plain functions called outside the injection context. At SDE-3 level, you should know how to use `runInInjectionContext` when you need DI in places that don't naturally have it.

Q: How do multi-providers enable a plugin architecture?

A: Multi-providers let you register multiple implementations under the same token. When injected, you get an array of all implementations. This is a plugin pattern: the consuming code iterates over all plugins without knowing which ones are registered. Angular uses this internally for interceptors (`HTTP_INTERCEPTORS`), app initializers (`APP_INITIALIZER`), and route guards. You can build your own — validation plugins, analytics trackers, feature

extensions — all registered via `multi: true`.

26. Custom RxJS Operators

Theory

SDE-3 developers don't just use built-in RxJS operators — they write reusable custom operators that encapsulate business logic, error handling, and caching patterns. A custom operator is a function that takes an Observable and returns an Observable — a higher-order function.

Hands-on Code

Custom operator — retry with exponential backoff

```
import { Observable, timer, throwError } from 'rxjs';
import { mergeMap, retryWhen, scan } from 'rxjs/operators';

export function retryWithBackoff<T>({
  maxRetries: number = 3,
  initialDelay: number = 1000,
  maxDelay: number = 30000
}) {
  return (source: Observable<T>): Observable<T> =>
    source.pipe(
      retryWhen(errors =>
        errors.pipe(
          scan((retryCount, error) => {
            if (retryCount >= maxRetries) throw error;
            return retryCount + 1;
          }, 0),
          mergeMap(retryCount => {
            const delay = Math.min(initialDelay * Math.pow(2, retryCount - 1), maxDelay);
            console.log(`Retry #${retryCount} after ${delay}ms`);
            return timer(delay);
          })
        )
      )
    );
}

// Usage:
this.http.get<Data>('/api/unstable-endpoint').pipe(
  retryWithBackoff(3, 1000, 10000)
).subscribe(data => this.data.set(data));
```

Custom operator — debug logger

```
export function debug<T>(tag: string) {
  return (source: Observable<T>): Observable<T> =>
    source.pipe(
      tap({
        next: val => console.log(`[${tag}] next:`, val),
        error: err => console.error(`[${tag}] error:`, err),
        complete: () => console.log(`[${tag}] complete`),
        subscribe: () => console.log(`[${tag}] subscribed`),
        unsubscribe: () => console.log(`[${tag}] unsubscribed`),
      })
    );
}

// Usage:
this.searchControl.valueChanges.pipe(
  debug('search-input'),
  debounceTime(300),
  distinctUntilChanged(),
  debug('after-debounce'),
  switchMap(term => this.api.search(term))
).subscribe();
```

Custom operator — cache with TTL

```
import { Observable, of, shareReplay, timer, switchMap, startWith } from 'rxjs';

export function cacheWithTTL<T>(ttl: number) {
  let cache$: Observable<T> | null = null;
  let lastFetch = 0;

  return (source: Observable<T>): Observable<T> => {
    const now = Date.now();
    if (!cache$ || now - lastFetch > ttl) {
      lastFetch = now;
      cache$ = source.pipe(shareReplay(1));
    }
    return cache$;
  };
}

// Usage — cache categories for 5 minutes
getCategories(): Observable<Category[]> {
  return this.http.get<Category[]>('/api/categories').pipe(
    cacheWithTTL(5 * 60 * 1000)
  );
}
```

Custom operator — loading state wrapper

```
export interface LoadingState<T> {
  loading: boolean;
  data: T | null;
  error: string | null;
}

export function withLoadingState<T>() {
  return (source: Observable<T>): Observable<LoadingState<T>> =>
    source.pipe(
      map(data => ({ loading: false, data, error: null } as LoadingState<T>)),
      startWith({ loading: true, data: null, error: null } as LoadingState<T>),
      catchError(err => of({
        loading: false,
        data: null,
        error: err?.message ?? 'Unknown error'
      } as LoadingState<T>))
    );
}

// Usage:
userState$ = this.userService.getUser(id).pipe(withLoadingState());
// Template:
// @if (userState$ | async; as state) {
//   @if (state.loading) { <spinner /> }
//   @if (state.error) { <error [message]="state.error" /> }
//   @if (state.data; as user) { <user-card [user]="user" /> }
// }
```

Interview Questions

Q: How do you write a custom RxJS operator?

A: A custom operator is a function that returns `(source: Observable<T>) => Observable<R>`. Inside, you pipe the source through existing operators to compose new behavior. Keep operators pure — no side effects or shared mutable state (except intentional caching). Type the generics properly so the operator is reusable. Test operators independently using marble testing.

Q: What is the difference between a pipeable operator and a creation operator?

A: Pipeable operators transform an existing observable (`map`, `filter`, `switchMap` — used inside `.pipe()`). Creation operators create a new observable from scratch (`of`, `from`, `interval`, `timer`, `fromEvent`). Custom operators are almost always pipeable — they receive a source and return a transformed observable.

27. Angular CDK (Component Dev Kit)

Theory

The CDK provides unstyled, behavior-only primitives for building UI components. At SDE-3 level, you use CDK instead of reinventing drag-drop, virtual scroll, overlays, and accessibility utilities.

Key CDK modules

Module	What it provides	When to use
cdk/overlay	Positioning engine for popups, tooltips, dropdowns	Custom dropdown menus, context menus, popovers
cdk/scrolling	Virtual scrolling — renders only visible items	Lists with 1000+ items (product lists, logs, timelines)
cdk/drag-drop	Drag & drop with reordering, transfer between lists	Kanban boards, sortable lists, drag-to-organize
cdk/a11y	FocusTrap, FocusMonitor, LiveAnnouncer, ListKeyManager	Dialogs, custom widgets, keyboard navigation
cdk/portal	Dynamically render components/templates into any DOM location	Modals, notification toasts, tooltips
cdk/clipboard	Copy text to clipboard	"Copy to clipboard" buttons
cdk/layout	BreakpointObserver — responsive design in TypeScript	Show/hide sidebar based on screen size

Hands-on Code

Virtual scrolling — rendering 10,000 items

```
import { ScrollingModule } from '@angular/cdk/scrolling';

@Component({
  imports: [ScrollingModule],
  template: `
    <cdk-virtual-scroll-viewport itemSize="48" class="list-viewport">
      <div *cdkVirtualFor="let item of items; trackBy: trackById"
        class="list-item">
        {{ item.name }} – {{ item.email }}
      </div>
    </cdk-virtual-scroll-viewport>
  `,
  styles: [`
    .list-viewport { height: 500px; }
    .list-item { height: 48px; display: flex; align-items: center; padding: 0 16px; }
  `]
})
export class UserListComponent {
  items = signal<User[]>([]); // 10,000 users
  trackById = (_, number, item: User) => item.id;

  constructor() {
    inject(UserService).getAll().subscribe(users => this.items.set(users));
  }
}
// Only ~15 items are in the DOM at any time, regardless of total count
```

Drag and drop — Kanban board

```
import { CdkDragDrop, DragDropModule, moveItemInArray, transferArrayItem } from '@angular/cdk/drag-drop';

@Component({
  imports: [DragDropModule],
  template: `
    <div class="board">
      <div class="column" cdkDropList [cdkDropListData]="todo()"
        (cdkDropListDropped)="drop($event)" id="todo-list"
        [cdkDropListConnectedTo]="['inprogress-list', 'done-list']">
        <h3>To Do</h3>
        @for (task of todo(); track task.id) {
          <div class="card" cdkDrag>{{ task.title }}</div>
        }
      </div>

      <div class="column" cdkDropList [cdkDropListData]="inProgress()"
        (cdkDropListDropped)="drop($event)" id="inprogress-list"
        [cdkDropListConnectedTo]="['todo-list', 'done-list']">
        <h3>In Progress</h3>
        @for (task of inProgress(); track task.id) {
          <div class="card" cdkDrag>{{ task.title }}</div>
        }
      </div>

      <div class="column" cdkDropList [cdkDropListData]="done()"
        (cdkDropListDropped)="drop($event)" id="done-list"
        [cdkDropListConnectedTo]="['todo-list', 'inprogress-list']">
        <h3>Done</h3>
        @for (task of done(); track task.id) {
          <div class="card" cdkDrag>{{ task.title }}</div>
        }
      </div>
    </div>
  `
})
export class KanbanBoardComponent {
  todo = signal<Task[]>([[{ id: '1', title: 'Design API' }]]);
  inProgress = signal<Task[]>([[{ id: '2', title: 'Build auth' }]]);
  done = signal<Task[]>([]);

  drop(event: CdkDragDrop<Task[]>) {
    if (event.previousContainer === event.container) {
      moveItemInArray(event.container.data, event.previousIndex, event.currentIndex);
    } else {
      transferArrayItem(event.previousContainer.data, event.container.data,
        event.previousIndex, event.currentIndex);
    }
  }
}
```

Overlay — custom dropdown

```
import { Overlay, OverlayRef, OverlayModule } from '@angular/cdk/overlay';
import { CdkPortal, PortalModule } from '@angular/cdk/portal';

@Component({
  imports: [OverlayModule, PortalModule],
  template: `
    <button #trigger (click)="toggle()">Open Menu</button>

    <ng-template cdkPortal #menuPortal="cdkPortal">
      <div class="dropdown-menu">
        <button (click)="onAction('edit')">Edit</button>
        <button (click)="onAction('delete')">Delete</button>
        <button (click)="onAction('share')">Share</button>
      </div>
    </ng-template>
  `
})
```

```

        </div>
      </ng-template>
    )
  })
  export class DropdownComponent {
    @ViewChild('trigger') trigger!: ElementRef;
    @ViewChild('menuPortal') menuPortal!: CdkPortal;
    private overlay = inject(Overlay);
    private overlayRef: OverlayRef | null = null;

    toggle() {
      if (this.overlayRef?.hasAttached()) {
        this.overlayRef.detach();
        return;
      }
      this.overlayRef = this.overlay.create({
        positionStrategy: this.overlay.position()
          .flexibleConnectedTo(this.trigger)
          .withPositions([
            { originX: 'start', originY: 'bottom', overlayX: 'start', overlayY: 'top' },
          ]),
        hasBackdrop: true,
        backdropClass: 'cdk-overlay-transparent-backdrop'
      });
      this.overlayRef.attach(this.menuPortal);
      this.overlayRef.backdropClick().subscribe(() => this.overlayRef?.detach());
    }

    onAction(action: string) {
      console.log(action);
      this.overlayRef?.detach();
    }
  }
}

```

Interview Questions

Q: When do you use CDK virtual scrolling vs pagination?

A: Virtual scrolling when: the user needs to scroll through a long list with a smooth experience, the data is already loaded or streamed, and you want "infinite scroll" feel. Pagination when: the data is paginated on the server (you can't fetch 10K records), the user needs to deep-link to a specific page, or SEO requires discrete pages. In practice, many SDE-3 solutions combine both: paginated API + virtual scroll within each page for smooth rendering.

28. Error Boundaries & Application Resilience

Theory

Production apps must be resilient. SDE-3 engineers design error handling strategies that prevent a single API failure from crashing the entire page.

Hands-on Code

Global error handler

```
@Injectable()
export class GlobalErrorHandler implements ErrorHandler {
  private errorService = inject(ErrorTrackingService);
  private notifier = inject(NotificationService);

  handleError(error: unknown) {
    // 1. Log to tracking service (Sentry, Datadog)
    this.errorService.capture(error);

    // 2. Show user-friendly notification
    const message = error instanceof HttpResponse
      ? `Server error: ${error.status}`
      : 'An unexpected error occurred';
    this.notifier.showError(message);

    // 3. Still log to console in dev
    console.error('GlobalErrorHandler:', error);
  }
}

// Register
export const appConfig: ApplicationConfig = {
  providers: [
    { provide: ErrorHandler, useClass: GlobalErrorHandler }
  ]
};
```

Circuit breaker pattern

```
@Injectable({ providedIn: 'root' })
export class CircuitBreaker {
  private failures = new Map<string, { count: number; openedAt: number }>();
  private threshold = 5;
  private resetTimeout = 30000; // 30 seconds

  canRequest(key: string): boolean {
    const state = this.failures.get(key);
    if (!state) return true;
    if (state.count < this.threshold) return true;
    // Circuit is OPEN – check if reset timeout has passed
    if (Date.now() - state.openedAt > this.resetTimeout) {
      this.failures.delete(key); // half-open: allow one request
      return true;
    }
    return false; // Circuit still open – reject
  }

  recordFailure(key: string) {
    const state = this.failures.get(key) ?? { count: 0, openedAt: 0 };
    state.count++;
    if (state.count >= this.threshold) state.openedAt = Date.now();
    this.failures.set(key, state);
  }

  recordSuccess(key: string) {
```

```

        this.failures.delete(key);
    }
}

// Usage in a service
getData(endpoint: string): Observable<Data> {
    if (!this.circuitBreaker.canRequest(endpoint)) {
        return throwError(() => new Error('Service temporarily unavailable'));
    }
    return this.http.get<Data>(endpoint).pipe(
        tap(() => this.circuitBreaker.recordSuccess(endpoint)),
        catchError(err => {
            this.circuitBreaker.recordFailure(endpoint);
            return throwError(() => err);
        })
    );
}

```

Graceful degradation with @defer @error

```

// Each section degrades independently – one failure doesn't crash the page
@defer (on idle) {
    <app-analytics-dashboard [data]="analyticsData()" />
} @error {
    <div class="degraded-widget">
        Analytics temporarily unavailable.
        <button (click)="retryAnalytics()">Retry</button>
    </div>
}

@defer (on viewport) {
    <app-recommendations [userId]="userId()" />
} @error {
    <p>Recommendations unavailable.</p>
}

```

Interview Questions

Q: How do you design resilient frontend applications?

A: (1) Global error handler that tracks errors without crashing the app. (2) HTTP interceptor with retry + exponential backoff. (3) Circuit breaker to stop hammering a failing API. (4) Graceful degradation — each widget/section fails independently. (5) Offline detection with `navigator.onLine` and service-worker fallbacks. (6) Error boundaries at the route level — a failing lazy module doesn't crash the shell. (7) Feature flags to disable broken features remotely without a deploy.

29. Web Workers & Off-main-thread Processing

Theory

The main thread handles rendering, user input, and JavaScript execution. Heavy computations (data processing, sorting 10K records, crypto, parsing) block the UI and degrade INP (Interaction to Next Paint). Web Workers run JavaScript on a separate thread, keeping the UI responsive.

Worker types

Type	Scope	Use case
Dedicated Worker	Single page/component	Heavy computation for one component
Shared Worker	Multiple tabs/windows	Shared WebSocket connection, cross-tab sync
Service Worker	Entire origin	Offline cache, push notifications, background sync

Hands-on Code

Angular CLI web worker

```
// Generate: ng generate web-worker heavy-computation

// heavy-computation.worker.ts
addEventListener('message', ({ data }) => {
  const { records, sortField, filterTerm } = data;

  // Heavy work – runs on separate thread, doesn't block UI
  let result = records;

  if (filterTerm) {
    result = result.filter((r: any) =>
      Object.values(r).some(v =>
        String(v).toLowerCase().includes(filterTerm.toLowerCase())
      )
    );
  }

  result.sort((a: any, b: any) => {
    const aVal = a[sortField];
    const bVal = b[sortField];
    return aVal < bVal ? -1 : aVal > bVal ? 1 : 0;
  });

  postMessage({ filteredRecords: result, total: result.length });
});

// Component using the worker
@Component({ ... })
export class DataTableComponent {
  private worker?: Worker;
  filteredData = signal<Record[]>([]);

  constructor() {
    if (typeof Worker !== 'undefined') {
      this.worker = new Worker(
        new URL('./heavy-computation.worker', import.meta.url)
      );
      this.worker.onmessage = ({ data }) => {
```

```

        this.filteredData.set(data.filteredRecords);
    };
    }
}

onFilterChange(term: string) {
    this.worker?.postMessage({
        records: this.rawData(),
        sortField: this.sortField(),
        filterTerm: term
    });
}
}

```

Comlink — making workers feel like regular functions

```

// worker.ts (with Comlink)
import * as Comlink from 'comlink';

const api = {
    processLargeDataset(data: number[]): number[] {
        // CPU-intensive – runs off main thread
        return data.map(x => Math.sqrt(x * x + Math.sin(x) * 1000));
    },

    async generateReport(records: Record[]): Promise<ReportResult> {
        // Complex aggregation
        const grouped = records.reduce(/* ... */);
        return { summary: grouped, generatedAt: Date.now() };
    }
};

Comlink.expose(api);

// Component – call worker methods like normal async functions
const worker = new Worker(new URL('./worker', import.meta.url));
const api = Comlink.wrap<typeof import('./worker').api>(worker);
const result = await api.processLargeDataset(hugeArray);

```

Interview Questions

Q: When should you use Web Workers in Angular?

A: When computation takes >50ms on the main thread (blocks a frame) — sorting/filtering large datasets, JSON parsing large payloads, crypto hashing, image processing, CSV/Excel export generation, complex validation. Do NOT use workers for simple operations — the message serialization overhead outweighs the benefit. Workers cannot access the DOM, so they are for pure data processing only.

30. WebSockets & Real-time Communication

Theory

Technology	Protocol	Direction	Use case
HTTP Polling	HTTP	Client → Server	Simple, works everywhere, high latency
Server-Sent Events (SSE)	HTTP	Server → Client only	Live feeds, notifications, stock prices
WebSocket	WS/WSS	Bidirectional	Chat, collaborative editing, gaming

Hands-on Code

WebSocket service with RxJS

```
@Injectable({ providedIn: 'root' })
export class WebSocketService {
  private socket$ = new WebSocketSubject<WsMessage>({
    url: 'wss://api.example.com/ws',
    openObserver: { next: () => console.log('WS connected') },
    closeObserver: { next: () => console.log('WS disconnected') }
  });

  // Auto-reconnect wrapper
  readonly messages$ = this.socket$.pipe(
    retryWhen(errors => errors.pipe(
      tap(err => console.warn('WS error, reconnecting...', err)),
      delay(3000)
    )),
    share()
  );

  // Type-safe message streams
  chatMessages$ = this.messages$.pipe(
    filter(msg => msg.type === 'chat'),
    map(msg => msg.payload as ChatMessage)
  );

  notifications$ = this.messages$.pipe(
    filter(msg => msg.type === 'notification'),
    map(msg => msg.payload as AppNotification)
  );

  send(message: WsMessage) {
    this.socket$.next(message);
  }

  close() {
    this.socket$.complete();
  }
}
```

Server-Sent Events with RxJS

```
@Injectable({ providedIn: 'root' })
export class SSEService {
  connect(url: string): Observable<MessageEvent> {
    return new Observable(observer => {
      const eventSource = new EventSource(url);
```

```

        eventSource.onmessage = (event) => observer.next(event);
        eventSource.onerror = (error) => observer.error(error);

        // Cleanup on unsubscribe
        return () => eventSource.close();
    });
}

// Usage – live stock prices
stockPrices$ = this.sse.connect('/api/stocks/stream').pipe(
    map(event => JSON.parse(event.data) as StockPrice),
    scan((acc, price) => ({ ...acc, [price.symbol]: price }), {}, { } as Record<string, StockPrice>)
);

```

Interview Questions

Q: When do you use WebSocket vs SSE vs polling?

A: WebSocket: bidirectional real-time (chat, multiplayer, collaborative editing). **SSE:** server-to-client push (live feeds, dashboards, notifications) — simpler than WebSocket, auto-reconnects, uses HTTP. **Polling:** when you need simplicity and can tolerate latency (checking for new email every 30s). At SDE-3 level, also consider: connection limits per domain (6 for HTTP/1.1 — SSE uses one), scaling considerations (WebSocket connections are stateful and harder to load-balance than HTTP).

31. Internationalization & Localization (i18n / l10n)

Theory

Enterprise apps serve users in multiple languages and regions. Angular has built-in i18n support (compile-time) and the ecosystem offers runtime alternatives (Transloco, ngx-translate). SDE-3 engineers choose the right approach and design the translation workflow.

Approaches compared

Approach	How it works	Pros	Cons
Angular i18n (built-in)	Compile-time — builds separate bundle per locale	Best performance, AOT-optimized, plural/ICU support	One build per language, can't switch at runtime
Transloco	Runtime — loads JSON translation files on demand	Switch language at runtime, lazy-load translations, great DX	Slightly larger runtime overhead
ngx-translate	Runtime — similar to Transloco but older	Well-established, large community	Less maintained, missing features vs Transloco

Hands-on Code

Transloco — runtime i18n setup

```
// transloco-root.module.ts
import { provideTransloco, TranslocoModule } from '@jsverse/transloco';

export const appConfig: ApplicationConfig = {
  providers: [
    provideTransloco({
      config: {
        availableLangs: ['en', 'hi', 'de', 'ja'],
        defaultLang: 'en',
        fallbackLang: 'en',
        reRenderOnLangChange: true,
        prodMode: !isDevMode()
      },
      loader: TranslocoHttpLoader // Loads /assets/i18n/en.json, hi.json, etc.
    })
  ]
};

// assets/i18n/en.json
{
  "dashboard": {
    "title": "Dashboard",
    "welcome": "Welcome, {{ name }}!",
    "items": "You have {{ count }} item(s)"
  }
}

// assets/i18n/hi.json
{
  "dashboard": {
    "title": "■■■■■■■■■■",
    "welcome": "■■■■■■■■ ■■, {{ name }}!",
    "items": "■■■■■ ■■■ {{ count }} ■■■■ ■■■■"
  }
}
```

```
// Template usage
<h1>{{ 'dashboard.title' | transloco }}</h1>
<p>{{ 'dashboard.welcome' | transloco:{ name: userName() } }}</p>

// Language switcher
<select (change)="switchLang($event)">
  @for (lang of availableLangs; track lang) {
    <option [value]="lang">{{ lang | uppercase }}</option>
  }
</select>
```

Angular built-in i18n — compile-time

```
<!-- Mark text for extraction -->
<h1 i18n="@@dashboardTitle">Dashboard</h1>
<p i18n="@@welcomeMessage">Welcome, {{ userName }}!</p>

<!-- ICU expressions for pluralization -->
<p i18n>
  {itemCount, plural,
    =0 {No items}
    =1 {One item}
    other {{{itemCount}} items}
  }
</p>

<!-- Extract: ng extract-i18n --output-path src/locale -->
<!-- Build: ng build --localize -->
```

Interview Questions

Q: How do you decide between compile-time and runtime i18n?

A: Compile-time (Angular i18n) when: you have a small number of locales (<5), need maximum performance, and your deployment pipeline can handle multiple builds. Runtime (Transloco) when: you need language switching without reload, have many locales, want lazy-loaded translations per route, or need to add new languages without rebuilding. Most enterprise SPA apps prefer runtime (Transloco) for flexibility.

32. Migration Strategies

Theory

SDE-3 engineers are expected to lead migrations — not just write code, but plan the strategy, manage risk, and execute incrementally without breaking production.

32.1 Common migration paths

Migration	Strategy	Key decisions
NgModules → Standalone	Incremental — use <code>ng generate @angular/core:standalone schematic</code>	Start with leaf components, work upward. Can coexist — standalone components can import NgModules via <code>importProvidersFrom</code>
Zone.js → Zoneless	Gradual — convert components to use signals, then enable <code>provideExperimentalZonelessChangeDetection()</code>	All change detection must be signal-driven. Async pipe auto-triggers CD in zoneless. Third-party libs that depend on Zone.js need evaluation.
<code>*ngIf/*ngFor</code> → <code>@if/@for</code>	Automated — use <code>ng generate @angular/core:control-flow migration schematic</code>	One-shot migration. Verify templates after migration. <code>@for</code> requires <code>track</code> expression.
Class-based guards → Functional	Replace <code>canActivate: [AuthGuard]</code> with <code>canActivate: [authGuard]</code>	Functional guards use <code>inject()</code> — simple functions, no class needed
AngularJS (1.x) → Angular	<code>ngUpgrade</code> — hybrid mode. Or strangler fig pattern with micro-frontends	Incremental route-by-route migration. New features in Angular, old pages in AngularJS. Eventually remove AngularJS.

32.2 Migration principles

- **Incremental, not big-bang:** Migrate one module/feature at a time. Ship to production after each increment.
- **Coexistence:** Old and new patterns must work side by side during the transition.
- **Automation:** Use Angular schematics for automated migrations wherever possible.
- **Testing gate:** No migration PR without tests passing. Add tests for migrated code if they don't exist.
- **Rollback plan:** Every migration step should be revertable with a git revert.
- **Communication:** Document the migration plan, share with the team, get buy-in before starting.

Hands-on Code

NgModule to standalone — step by step

```
// BEFORE: NgModule-based
@NgModule({
  declarations: [ProductListComponent, ProductCardComponent],
  imports: [CommonModule, RouterModule],
  exports: [ProductListComponent]
})
export class ProductModule {}
```

```
// STEP 1: Convert leaf component to standalone
@Component({
  selector: 'app-product-card',
  standalone: true,
  imports: [CommonModule, CurrencyPipe],
  template: `...`
})
export class ProductCardComponent { ... }

// STEP 2: Convert parent component
@Component({
  selector: 'app-product-list',
  standalone: true,
  imports: [CommonModule, RouterModule, ProductCardComponent],
  template: `...`
})
export class ProductListComponent { ... }

// STEP 3: Remove the NgModule (it's no longer needed)
// STEP 4: Update route config to use loadComponent instead of loadChildren
```

Interview Questions

Q: How would you migrate a large Angular app from NgModules to standalone?

A: (1) Run the Angular schematic as a starting point. (2) Start with leaf components (no children) — they have the fewest dependencies. (3) Convert bottom-up — children before parents. (4) Use `importProvidersFrom(ExistingModule)` as a bridge during transition. (5) Update route configs from `loadChildren` to `loadComponent`. (6) Remove empty NgModules after all their declarations are standalone. (7) Do it in small PRs — each PR converts one module, runs full CI, and ships to production.

Q: What is the strangler fig pattern and how does it apply to frontend migration?

A: The strangler fig pattern incrementally replaces an old system by building new features alongside it, then gradually routing traffic from old to new. For AngularJS → Angular: run both frameworks in the same app (`ngUpgrade`) or use micro-frontends (old routes served by AngularJS, new routes by Angular). Over time, migrate route by route until AngularJS is completely removed. Never do a big-bang rewrite — it takes too long and has too much risk.

33. Advanced Testing Patterns

Theory

At SDE-3 level, you move beyond basic unit tests to marble testing for RxJS, component test harnesses, testing interceptors, testing guards, and establishing testing strategy for the team.

Hands-on Code

Marble testing for RxJS streams

```
import { TestScheduler } from 'rxjs/testing';

describe('SearchService', () => {
  let scheduler: TestScheduler;

  beforeEach(() => {
    scheduler = new TestScheduler((actual, expected) => {
      expect(actual).toEqual(expected);
    });
  });

  it('should debounce and deduplicate search terms', () => {
    scheduler.run(({ cold, expectObservable }) => {
      // Simulate user typing: 'a' at 0ms, 'ab' at 10ms, 'abc' at 20ms
      const input$ = cold('a-b-c---|', { a: 'a', b: 'ab', c: 'abc' });
      const expected = '-----c|'; // Only 'abc' after debounce

      const result$ = input$.pipe(
        debounceTime(30, scheduler),
        distinctUntilChanged()
      );

      expectObservable(result$).toBe(expected, { c: 'abc' });
    });
  });

  // Theory: Marble testing lets you test time-dependent RxJS streams
  // synchronously using virtual time. The TestScheduler replaces real timers
  // with virtual ones, so debounceTime(300) resolves instantly in tests.
});
```

Testing an HTTP interceptor

```
describe('authInterceptor', () => {
  let httpMock: HttpTestingController;
  let httpClient: HttpClient;

  beforeEach(() => {
    TestBed.configureTestingModule({
      providers: [
        provideHttpClient(withInterceptors([authInterceptor])),
        provideHttpClientTesting(),
        { provide: AuthService, useValue: { getToken: () => 'test-jwt-token' } }
      ]
    });

    httpClient = TestBed.inject(HttpClient);
    httpMock = TestBed.inject(HttpTestingController);
  });

  it('should add Authorization header', () => {
    httpClient.get('/api/data').subscribe();

    const req = httpMock.expectOne('/api/data');
    expect(req.request.headers.get('Authorization')).toBe('Bearer test-jwt-token');
```

```

    req.flush({ data: 'test' });
  });

it('should not add header when no token', () => {
  TestBed.overrideProvider(AuthService, {
    useValue: { getToken: () => null }
  });

  httpClient.get('/api/data').subscribe();
  const req = httpMock.expectOne('/api/data');
  expect(req.request.headers.has('Authorization')).toBe(false);
  req.flush({ data: 'test' });
});

afterEach(() => httpMock.verify()); // Ensure no unmatched requests
});

```

Component test harness (Angular CDK)

```

// Creating a harness for a custom component
export class ProductCardHarness extends ComponentHarness {
  static hostSelector = 'app-product-card';

  // Locators
  private getTitle = this.locatorFor('.product-title');
  private getPrice = this.locatorFor('.product-price');
  private getAddButton = this.locatorFor('button.add-to-cart');

  async getProductName(): Promise<string> {
    return (await this.getTitle()).text();
  }

  async getProductPrice(): Promise<string> {
    return (await this.getPrice()).text();
  }

  async clickAddToCart(): Promise<void> {
    return (await this.getAddButton()).click();
  }

  async isAddButtonDisabled(): Promise<boolean> {
    const btn = await this.getAddButton();
    return (await btn.getAttribute('disabled')) !== null;
  }
}

// Usage in tests
it('should display product and allow add to cart', async () => {
  const loader = TestBedHarnessEnvironment.loader(fixture);
  const card = await loader.getHarness(ProductCardHarness);

  expect(await card.getProductName()).toBe('Laptop');
  expect(await card.getProductPrice()).toContain('999');

  await card.clickAddToCart();
  expect(cardServiceSpy.add).toHaveBeenCalledWith(jasmine.objectContaining({ name: 'Laptop' }));
});

```

Testing a functional guard

```

describe('authGuard', () => {
  it('should allow authenticated users', () => {
    TestBed.configureTestingModule({
      providers: [
        { provide: AuthService, useValue: { isLoggedIn: () => true } },
        provideRouter([
          { path: 'test', component: DummyComponent, canActivate: [authGuard] }
        ])
      ]
    });
  });
});

```



```

    });

    const router = TestBed.inject(Router);
    const result = TestBed.runInInjectionContext(() =>
      authGuard({} as ActivatedRouteSnapshot, { url: '/test' } as RouterStateSnapshot)
    );

    expect(result).toBe(true);
  });

  it('should redirect unauthenticated users to login', () => {
    TestBed.configureTestingModule({
      providers: [
        { provide: AuthService, useValue: { isLoggedIn: () => false } },
        provideRouter([])
      ]
    });

    const result = TestBed.runInInjectionContext(() =>
      authGuard({} as ActivatedRouteSnapshot, { url: '/dashboard' } as RouterStateSnapshot)
    );

    expect(result).toBeInstanceOf(UrlTree);
  });
});

```

Interview Questions

Q: What is marble testing and when do you use it?

A: Marble testing is a technique for testing RxJS observables using ASCII marble diagrams that represent values over virtual time. `---a--b--|` means: value 'a' at frame 3, 'b' at frame 6, complete at frame 9. It is essential for testing time-dependent operators (`debounceTime`, `delay`, `timeout`, `switchMap`) without real timers. Without marble testing, you'd need `setTimeout` in tests, making them slow and flaky.

Q: What are component test harnesses and why use them?

A: Test harnesses are an abstraction layer over component DOM queries. Instead of using `querySelector('.product-title')` (brittle — breaks when CSS changes), you use `harness.getProductName()` (stable API). Angular Material ships harnesses for all its components. Building harnesses for your own components lets consumers (other teams, E2E tests) interact with your components through a stable API. This is especially important in a design system consumed by multiple teams.

34. CI/CD Pipelines for Angular

Theory

SDE-3 engineers own the CI/CD pipeline for the team. You define what runs, in what order, and how to keep it fast.

Hands-on Code

Complete GitHub Actions pipeline

```
name: Angular CI/CD

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true # Cancel previous runs for the same branch

jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }

      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: 'npm' }

      - run: npm ci

      # Lint – catches code style and module boundary violations
      - run: npx nx affected -t lint --parallel=3

      # Unit tests – with code coverage
      - run: npx nx affected -t test --parallel=3 --code-coverage
        env:
          CI: true

      # Build – ensures production build succeeds
      - run: npx nx affected -t build --configuration=production

      # Bundle size check
      - uses: andresz1/size-limit-action@v1
        with:
          github_token: ${{ secrets.GITHUB_TOKEN }}

  e2e:
    needs: quality
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: 'npm' }
      - run: npm ci
      - run: npx nx affected -t e2e --parallel=1

  deploy:
    needs: [quality, e2e]
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```
- uses: actions/setup-node@v4
  with: { node-version: 20, cache: 'npm' }
- run: npm ci
- run: npx nx run my-app:build:production
- name: Deploy to CDN / Cloud
  run: |
    # Deploy dist/ to S3, Vercel, Firebase, etc.
    echo "Deploying..."
```

Interview Questions

Q: How do you optimize CI pipeline speed for a large Angular project?

A: (1) Use nx affected to only run tasks for changed code. (2) Enable Nx Cloud for distributed computation caching. (3) Parallelize independent tasks (lint, test, build). (4) Use concurrency groups to cancel redundant runs. (5) Cache node_modules between runs. (6) Run E2E tests only on main branch or as a separate slower pipeline. (7) Set bundle budgets so size regressions fail fast. A well-tuned pipeline for a large monorepo should complete in <10 minutes for PRs.

35. Observability & Monitoring

Theory

SDE-3 engineers don't just build features — they ensure the app is observable in production. You need to know what is happening, what broke, and why.

Three pillars of frontend observability

Pillar	What it tracks	Tools
Error tracking	Unhandled errors, promise rejections, HTTP errors	Sentry, Datadog, Bugsnag
Performance monitoring (RUM)	Core Web Vitals, page load, API latency, INP	Sentry Performance, Datadog RUM, web-vitals lib
User analytics	Feature usage, navigation patterns, funnel drops	Amplitude, Mixpanel, PostHog

Hands-on Code

Sentry integration for Angular

```
// main.ts
import * as Sentry from '@sentry/angular';

Sentry.init({
  dsn: 'https://examplePublicKey@o0.ingest.sentry.io/0',
  integrations: [
    Sentry.browserTracingIntegration(),
    Sentry.replayIntegration()
  ],
  tracesSampleRate: 0.2, // 20% of transactions
  replaysSessionSampleRate: 0.1,
  environment: environment.name,
  release: environment.version
});

// app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    { provide: ErrorHandler, useValue: Sentry.createErrorHandler({ showDialog: false }) },
    { provide: Sentry.TraceService, deps: [Router] },
    { provide: APP_INITIALIZER, useFactory: () => () => {}, deps: [Sentry.TraceService], multi: true }
  ]
};
```

Custom performance monitoring service

```
@Injectable({ providedIn: 'root' })
export class PerformanceMonitorService {
  trackCoreWebVitals() {
    import('web-vitals').then(({ onLCP, onINP, onCLS, onFCP, onTTFB }) => {
      onLCP(metric => this.report('LCP', metric));
      onINP(metric => this.report('INP', metric));
      onCLS(metric => this.report('CLS', metric));
      onFCP(metric => this.report('FCP', metric));
      onTTFB(metric => this.report('TTFB', metric));
    });
  }

  private report(name: string, metric: { value: number; rating: string }) {
    // Send to analytics backend
  }
}
```

```

        navigator.sendBeacon('/api/metrics', JSON.stringify({
            name,
            value: metric.value,
            rating: metric.rating,    // 'good' | 'needs-improvement' | 'poor'
            url: location.href,
            timestamp: Date.now()
        }));
    }
}

// Usage in app.component.ts constructor:
inject(PerformanceMonitorService).trackCoreWebVitals();

```

Feature flags service

```

@Injectable({ providedIn: 'root' })
export class FeatureFlagService {
    private flags = signal<Record<string, boolean>>({});
    private http = inject(HttpClient);

    constructor() {
        // Load flags from server
        this.http.get<Record<string, boolean>>('/api/feature-flags').pipe(
            catchError(() => of({})) // Fail gracefully – all flags off
        ).subscribe(flags => this.flags.set(flags));
    }

    isEnabled(flag: string): boolean {
        return this.flags()[flag] ?? false;
    }

    // Reactive check for templates
    enabled(flag: string) {
        return computed(() => this.flags()[flag] ?? false);
    }
}

// Usage in template:
// @if (featureFlags.enabled('newCheckout')) {
//   <app-new-checkout />
// } @else {
//   <app-legacy-checkout />
// }
//
// Feature flags let you disable broken features in production
// WITHOUT a new deploy – critical for SDE-3 level ownership.

```

Interview Questions

Q: How do you set up error tracking in a production Angular app?

A: (1) Integrate Sentry/Datadog with a custom `ErrorHandler`. (2) Upload source maps during build (so stack traces show original TypeScript, not minified JS). (3) Set sample rates to control volume. (4) Add user context (user ID, email) for debugging. (5) Track HTTP errors via an interceptor. (6) Set up alerts for error spikes. (7) Use Session Replay to see what the user was doing when the error occurred.

Q: What are feature flags and how do they enable safe deployments?

A: Feature flags are runtime toggles that control which features are active. Benefits for SDE-3: (1) Ship code to production behind a flag before it's ready — trunk-based development. (2) Gradual rollout — enable for 5% of users, then 25%, then 100%. (3) Kill switch — disable a broken feature instantly without a deploy. (4) A/B testing — different users see different implementations. Services like LaunchDarkly, Split, or a custom API provide flag management.

36. Design System & Component Library

Theory

SDE-3 engineers often lead the creation of a shared component library consumed by multiple teams. This is different from building one-off components — it requires API design, documentation, versioning, and governance.

Key principles

- **Headless / unstyled primitives:** Build behavior-only components (like CDK) that consumers can style. Or provide a styled library with CSS custom properties for theming.
- **Composition over configuration:** Prefer `<card><card-header>...</card-header></card>` over `<card [title]="..." [subtitle]="..." [actions]="...">`. Composition via content projection is more flexible.
- **Accessibility by default:** Every component ships with proper ARIA, keyboard navigation, and focus management.
- **Storybook:** Visual documentation and testing for every component variant.
- **Semantic versioning:** Breaking API changes require a major version bump. Communicate deprecations early.
- **Changelog:** Auto-generated from conventional commits.
- **Component test harnesses:** Ship CDK harnesses so consumers can test their code without depending on your DOM structure.

Hands-on Code

Themeable component with CSS custom properties

```
@Component({
  selector: 'ds-button',
  standalone: true,
  template: `
    <button
      [class]="ds-btn ds-btn--' + variant()"
      [disabled]="disabled()"
      [attr.aria-busy]="loading() || null"
    >
      @if (loading()) {
        <span class="ds-btn__spinner" aria-hidden="true"></span>
      }
      <ng-content />
    </button>
  `,
  styles: [
    .ds-btn {
      /* Themeable via CSS custom properties */
      padding: var(--ds-btn-padding, 8px 16px);
      border-radius: var(--ds-btn-radius, 6px);
      font-size: var(--ds-btn-font-size, 14px);
      font-weight: var(--ds-btn-font-weight, 600);
      border: 1px solid transparent;
      cursor: pointer;
      transition: all 0.15s ease;
    }
    .ds-btn--primary {
      background: var(--ds-color-primary, #0f6b5f);
      color: var(--ds-color-on-primary, #fff);
    }
    .ds-btn--secondary {
      background: var(--ds-color-secondary, #e8e5de);
      color: var(--ds-color-on-secondary, #1d1d1b);
    }
  ]
})
```

```

    .ds-btn:disabled { opacity: 0.5; cursor: not-allowed; }
    .ds-btn__spinner {
      display: inline-block; width: 14px; height: 14px;
      border: 2px solid currentColor; border-top-color: transparent;
      border-radius: 50%; animation: spin 0.6s linear infinite;
    }
    @keyframes spin { to { transform: rotate(360deg); } }
  `]
})
export class DsButtonComponent {
  variant = input<'primary' | 'secondary' | 'ghost'>('primary');
  disabled = input(false, { transform: booleanAttribute });
  loading = input(false, { transform: booleanAttribute });
}

// Consumer can theme via CSS custom properties:
// :root { --ds-color-primary: #1a73e8; --ds-btn-radius: 20px; }

```

Storybook story for the button

```

// button.stories.ts
import type { Meta, StoryObj } from '@storybook/angular';
import { DsButtonComponent } from './button.component';

const meta: Meta<DsButtonComponent> = {
  title: 'Components/Button',
  component: DsButtonComponent,
  tags: ['autodocs'],
  argTypes: {
    variant: { control: 'radio', options: ['primary', 'secondary', 'ghost'] },
    disabled: { control: 'boolean' },
    loading: { control: 'boolean' }
  }
};
export default meta;

type Story = StoryObj<DsButtonComponent>;

export const Primary: Story = {
  args: { variant: 'primary' },
  render: (args) => ({
    props: args,
    template: `<ds-button [variant]="variant" [disabled]="disabled" [loading]="loading">Click me
  })
};

export const Loading: Story = {
  args: { variant: 'primary', loading: true },
  render: (args) => ({
    props: args,
    template: `<ds-button [variant]="variant" [loading]="loading">Submitting...</ds-button>`
  })
};

```

Interview Questions

Q: How do you design a component library that scales across teams?

A: (1) Start with design tokens (CSS custom properties for colors, spacing, typography). (2) Build unstyled primitives first, then styled variants. (3) Use composition (content projection) over configuration (too many inputs). (4) Ship test harnesses so consumers don't break when DOM changes. (5) Use Storybook for documentation and visual testing. (6) Semantic versioning with auto-generated changelogs. (7) Publish as a private npm package or Nx publishable library. (8) Set up visual regression testing (Chromatic) to catch unintended style changes.

Q: How do you make a component library themeable?

A: CSS custom properties (design tokens). Define tokens like `--ds-color-primary`, `--ds-radius-md`, `--ds-spacing-4`. Components read these tokens. Consumers override them at `:root` or any element level for nested themes (light/dark mode, brand variants). This works across Angular's view encapsulation because CSS custom properties inherit through the DOM tree. Avoid hardcoded colors or sizes inside the library.

37. Architecture Decision Records & Technical Leadership

Theory

At SDE-3, you are a technical leader — not just a coder. You make architecture decisions, document them, mentor juniors, and set standards for the team.

37.1 Architecture Decision Records (ADRs)

An ADR is a short document that captures a significant architectural decision, the context, the options considered, and the rationale for the chosen option.

```
// docs/adr/002-state-management-approach.md

# ADR-002: State Management Approach

## Status: Accepted
## Date: 2024-03-15

## Context
Our app has 40+ components sharing state across 8 feature modules.
Currently using ad-hoc services with BehaviorSubject, causing
inconsistent patterns and difficulty in debugging state changes.

## Options Considered
1. **Signal-based stores (custom)** – lightweight, Angular-native, no library
2. **NgRx Store** – Redux pattern, time-travel debugging, effects
3. **NgRx Signal Store** – signal-based, less boilerplate than NgRx Store
4. **Keep current approach** – refactor existing services

## Decision
Option 3: NgRx Signal Store

## Rationale
- Signals align with Angular's direction (zoneless future)
- Less boilerplate than classic NgRx Store (no actions/reducers)
- Built-in computed state and effects
- DevTools support for debugging
- Team already familiar with signals from recent migration
- NgRx ecosystem (effects, entity) available when needed

## Consequences
- Team needs 1-2 days of NgRx Signal Store training
- Existing BehaviorSubject services will be migrated incrementally
- New features must use Signal Store from day one
```

37.2 Code review leadership

- **Review for architecture, not just syntax:** Is the component doing too much? Is state in the right place? Is this testable?
- **Be constructive:** "Consider using switchMap here to avoid race conditions" > "This is wrong."
- **Set up review checklists:** Performance (OnPush?), accessibility (semantic HTML?), testing (coverage?), security (sanitization?).
- **Automate what you can:** Linting catches style issues. Bundle budgets catch size regressions. Let humans review design and logic.
- **Timebox reviews:** Review within 4 hours. Don't block the team.

37.3 Mentoring patterns

- **Pair programming on complex features** — share context, transfer knowledge live.
- **Architecture workshops** — walk the team through system design decisions.

- **"Why" documentation** — code tells you what, comments and ADRs tell you why.
- **Runbooks** — how to deploy, how to rollback, how to debug common production issues.

Interview Questions

Q: How do you make technical decisions for the team?

A: (1) Identify the problem clearly — what pain are we solving? (2) List options with pros/cons. (3) Build a small proof-of-concept if the decision is high-risk. (4) Discuss with the team — get input, not just buy-in. (5) Document the decision as an ADR (Architecture Decision Record). (6) Commit to the decision — avoid revisiting without new evidence. (7) Monitor the outcome — was the predicted benefit realized? ADRs prevent "why did we choose this?" conversations 6 months later.

Q: How do you handle tech debt in a fast-moving team?

A: (1) Track tech debt like any other work — add it to the backlog with impact and effort estimates. (2) Allocate 15-20% of each sprint to debt reduction. (3) Attach debt reduction to feature work — "while building feature X, we'll also refactor the related service." (4) Prioritize debt that slows the team (brittle tests, unclear module boundaries) over cosmetic debt. (5) Never let debt accumulate silently — visibility is the first step to managing it.

Q: Describe your approach to code reviews.

A: I review for: (1) Correctness — does it do what it should? (2) Architecture — is this the right abstraction? Is state in the right place? (3) Performance — OnPush, trackBy, subscription cleanup. (4) Testability — can this be unit tested without complex setup? (5) Security — user input handling, auth checks. I automate style/lint checks so human reviews focus on design. I aim to review within 4 hours and give specific, actionable feedback with code examples.

38. Updated SDE-3 Interview Readiness Checklist

SDE-3 specific expectations vs SDE-2

Dimension	SDE-2 (mid-level)	SDE-3 (senior)
Scope	Owens features within a module	Owens entire frontend architecture for a product
Decision making	Follows patterns set by seniors	Defines patterns, documents ADRs, evaluates tradeoffs
Code quality	Writes clean code	Sets coding standards, leads code reviews, builds shared tooling
Performance	Fixes performance bugs	Designs performance budgets, sets up RUM monitoring, prevents issues proactively
Testing	Writes unit tests	Defines testing strategy, sets up test infrastructure, marble tests, harnesses
Cross-team	Works within their team	Builds shared libraries consumed by multiple teams, defines APIs and contracts
Mentoring	Helps peers occasionally	Actively mentors juniors, pair programs, leads architecture workshops
Production	Deploys features	Owens CI/CD, monitoring, feature flags, incident response, rollback plans

SDE-3 topic priority for interviews

Priority	Topic	Section
■ Must	Advanced DI (multi-providers, injection context, scoped services)	§25
■ Must	Custom RxJS operators (backoff, cache, loading state)	§26
■ Must	Application resilience (error handler, circuit breaker, degradation)	§28
■ Must	Advanced testing (marble, interceptors, harnesses, guards)	§33
■ Must	Architecture decisions & technical leadership	§37
■ Should	Micro-frontend architecture	§23

Priority	Topic	Section
■ Should	Monorepo with Nx	§24
■ Should	Angular CDK (overlay, virtual scroll, drag-drop)	§27
■ Should	CI/CD pipeline design	§34
■ Should	Observability & monitoring	§35
■ Should	Design system & component library	§36
■ Good	Web Workers	§29
■ Good	WebSockets & SSE	§30
■ Good	Internationalization (i18n)	§31
■ Good	Migration strategies	§32